Troubleshooting

Every entry below is symptom -> check -> fix, grounded in the code that ships in this repo, not in how a hook is supposed to behave. Where a command is shown as `<crew>/hooks/scripts/...`, that is `${CLAUDE_PLUGIN_ROOT}/hooks/scripts/...` inside a command file, or the plugin's cache directory when you run it by hand — see the shorthand in Memory and Obsidian.

Stale Install

Symptom: a command from this guide (`"/crew:status"`, a new hook, a config key) does not exist, or behaves like an older release.

- **Check the installed version against the repo.**

```
cat ~/.claude/plugins/installed_plugins.json | grep -A3 '"crew"'
```

Compare the commit or version it records against `plugin/crew/.claude-plugin/plugin.json`'s `version` in the marketplace clone. `claude plugin update` decides whether to re-copy a plugin by comparing the **declared version**, not its content — a content change that ships with no version bump means every already-installed copy reports "already at the latest version" forever, holding the old code, with nothing in the repo itself looking wrong (`CLAUDE.md`, "Stop and ask"). This class of bug is why `/crew:status`'s absence, or a hook that clearly predates a documented fix, is worth checking here before assuming the fix is wrong.

A related, sharper case: on a `directory`-source install (a dev checkout registered as a local plugin path rather than pulled from a marketplace), `installed_plugins.json` can name a version string with **no relationship to the code that actually executes** — the tree runs whatever is on disk regardless of what that file says (`TODO.md`, "`installed_plugins.json` records 0.19.56 while 0.19.63 executes"). On that install shape, trust the code over the version string.

- **Fix:**

```
claude plugin update crew@useful-claude-add-ons
```

then **restart** `claude` — a running session does not pick up a re-copied plugin. If the update reports "already at the latest version" and you can see the fix in the marketplace repo's `main`, the version was not bumped when it should have been; that is a defect to file against the plugin, not something a client-side retry fixes.

- **Check it worked:**

```
/crew:status
```

Read-only, at most 40 lines, dispatches nothing. Its `config` line names the config file it read: `.crew/config.json` for a repository `/crew:init` set up, ending `run /crew:migrate` only when that file still holds a 0.20 setup, or `.crew/crew.json` once `/crew:migrate` has run; `codemap` reports `behind: <subsystem>` when a map's anchor and the path diff both say it is stale.

Hook Noise

Every hook crew runs, and the config key that turns its behaviour up or down. `plugin/crew/hooks/hooks.json` registers one bash launcher per event (`crew-hook.sh`; `crew-hook.ps1` is only Codex's `commandWindows`); the launcher starts one dispatcher, `crew_hook.py <Event>`, which runs that event's scripts below at once and combines what they print. On Windows the same bash launcher is the only registration and the dispatcher runs the hot guards' Python directly (set `CREW_HOOK_DIRECT=0` to go back to their `.sh` wrappers), so no event fires twice. When the launcher or dispatcher itself fails in a repo crew governs, `PreToolUse`, `UserPromptSubmit` and `Stop` refuse with `crew-hook: ... failing closed (armed repo)` and the cause, rather than let the call through unjudged. Only the four stateless PreToolUse judges (`role-write-guard`, `cloud-guard`, `env-guard`, `scope-guard`) are run once more after a timeout, and only when the dispatch deadline leaves room for a second timeout's cleanup; at the defaults that is `role-write-guard` on every platform and the 15-second `cloud-guard`, `env-guard` and `scope-guard` rows off Windows. `promote-gate`, `approval-hook`, `verify-gate` and `completion-audit` are never retried (they record state), and a retried guard can leave a duplicate `guard.log` row. The line says `attempts=N`, so you can tell whether a retry ran. If the guard still cannot answer, the refusal is led by `crew-hook: UNKNOWN (guard timeout, not a policy deny)`. A guard that could not fork or allocate (a stderr line that is itself `fork: ... Resource temporarily unavailable`, `OSError: [Errno 11]`, `BlockingIOError: EAGAIN`, a bare `MemoryError`; text that merely quotes those words, such as a path, is not matched) is led by `UNKNOWN (resource exhaustion, not a policy deny)`, and a guard's own "no usable python" refusal by `UNKNOWN (no usable python, not a policy deny)` (only that fixed one-line refusal; a deny that quotes the words in a path is still a deny). An `UNKNOWN` line is a refusal because crew could not tell, not because a rule said no: retry the call, and free memory or processes if it repeats. A real deny never carries the label. Setting `CREW_HOOK_TIMING_FILE` to a path ending `.crew-trace` records each guard's start, end and milliseconds.

Event	Script	What it does	Switch
SessionStart	handoff-read.sh	resets its once-per-session markers; reads back .work/HANDOFF.md only if the context hook did not inject it	— (always runs)
SessionStart	platform-sync.sh	detects/repairs platform.{os,wsl,shell,windowsHostIp}	— (always runs)
SessionStart / UserPromptSubmit / PostToolUse / SubagentStart	crew-context.sh	injects code-map slices and vault recall, tracks the token budget	context.enabled (wh hook; injection itse separate switch sin 1.0.0)
UserPromptSubmit	approval-hook.sh	records or refuses a /crew:approve <id> receipt	always records; enforcement deper scope.mode
PreToolUse (Bash/PowerShell)	promote-gate.sh	the six command/production guards plus mergeGate	guards.profile, or guards.overrides.<g for terraformApply, forcePush, adminMer mergeGate, cloudDes sqlDestructive, prodDatabase, prodS
PreToolUse (Write|Edit)	role-write-guard.sh	refuses a write outside the dispatched role's declared scope	guards.overrides.ro (block/report/off), guards.profile
PreToolUse (Bash|PowerShell)	cloud-guard.sh	destructive aws/az commands, wrong-identity commands, and dispatches of workflows listed in environments.workflows (guards.overrides.deployWorkflow) — judged only when every word on the line is a plain literal or a single-quoted word; anything else asks, and is refused unattended	guards.overrides.cl (block/report/off), guards.profile
PreToolUse (Bash|	env-guard.sh	a command that prints the whole environment or a	guards.overrides.en (block/report/off),

Event	Script	What it does	Switch
PowerShell)		credential-named variable into the agent's context; a command it cannot read is could-not-tell (L-0772)	guards.profile
PreToolUse (Bash|PowerShell)	cloud-guard.sh, environment layer (T-0005)	a terraform apply judged by its target environment, and a destroy never applied unattended	environments.nonPr only; globs naming production workspa environments.prodU (both layers; true o when **both** say tru
PreToolUse (Write|Edit|MultiEdit|NotebookEdit|Bash|PowerShell)	scope-guard.sh	plan-approval + ticket scope guard	scope.mode (off/report/block/au scope.allowCliAppr
PreCompact	handoff-write.sh	writes the handoff note before compaction	— (always on; the path is fixed at .work/HANDOFF.md)
Notification	notify.sh (crew_notify.py hook)	a Question / Needs permission ping when Claude stopped on a permission prompt, an AskUserQuestion or an elicitation; never idle_prompt; once per waiting episode	notify.provider (no disables it), notify.
Stop	verify-gate.sh	runs .crew/verify.json's checks	verifyGate (boolean
Stop	context-watch.sh	nags for a handoff near the context budget, drives auto-clear	context.enabled, context.warnAt, context.reserveTok context.autoClear.
Stop	completion-audit.sh	diffs the whole tree against the ticket's scope base	scope.mode

- **Blocker pings (`notify.events` has `blocker`).** `Stop gate refused` means the verify gate or the completion audit refused the same ticket at Stop twice in a row (`crew_notify.py stop` records it; the gates call it once their harness-only change

lands). `Lane stalled` means `/crew:autopilot` found another runner's in-flight marker `stale` (a dead pid, or no heartbeat inside T-0049's 30-minute TTL) and names the owner's `crew_inflight.py clear`; `Lane state unknown` means it could not tell. `Approval waiting` is autopilot stopped at `approve`.

- **Silence one hook without touching the rest:** set its own key. `guards.profile: off` (which leaves the role-write, cloud and environment guards off) and `scope.mode: off` are already the shipped defaults — a noisy session usually means one of these was turned on somewhere (repo or machine-global) and forgotten, not that the default changed.
- **Silence the SessionStart context specifically:** `{"context": {"enabled": false}}` stops the context hook and the Stop nag with it. There is no separate switch for injection since 1.0.0 (the `memory.inject` setting was removed).
- **See what the context hook is actually doing:**

```
python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats
```

Reports emissions, injected characters, dedup hits, and vault-recall hit/miss/skip counts with a reason breakdown (see "Obsidian" below). The log itself is `<git-common-dir>/crew/context-log.jsonl` (or `.work/crew/context-log.jsonl` outside git), shared across every worktree of the repo. - **A hook fails and you cannot tell why:** every wrapper here resolves its own Python (`python3`/`python`/`py`) rather than assuming one is present, and every one that can block fails **closed** — never silently — when no interpreter resolves. See "no-python fail-closed" under Scope, below; the same shape applies to `scope-guard.sh`, `role-write-guard.sh`, `cloud-guard.sh` and `verify-gate.sh`.

Review Loops

Each ticket gets **two** review rounds, reserved before the reviewer runs, tracked in `<git-common-dir>/crew/review/<ticket>.json` — shared by every worktree of the repo, so a second worktree of the same repo spends the same budget (`review_ledger.py`).

- **Symptom: review keeps re-running / "budget exhausted". Check:**

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --status
```

A round's outcome is `CLEAN`, `FINDINGS` or `INCOMPLETE`. A completed round 2 — either `FINDINGS` or `INCOMPLETE` — leaves the ticket state `REVIEWED`, so its `FINDINGS` can still be accepted. A **third** reservation attempt is refused outright and the state becomes `NEEDS_REPLAN`; that refusal, an explicit `--reject`, and `--reject --supersede-accepted` on an `ACCEPTED` ticket are the only ways into `NEEDS_REPLAN` (autopilot's `crew_autopilot.py auto-reject`, under

`autopilot.maxAutoReplans`, is a `--reject` by the name `autopilot (policy: autopilot.maxAutoReplans)`). **Fix:** a final round with 0 BLOCK from a Codex or Kimi reviewer closes itself: `review: auto-accept: eligible`, then `--auto-accept --follow-up <id>` writes an `auto-accepted` receipt and its FIX/NIT lines go verbatim into one follow-up ticket. A `review: auto-accept: refused - <reason>` line names what stopped it (any BLOCK, INCOMPLETE, not the final round, an open healer skip, a count it could not read, a same-family Claude-fallback round, a provider or family it could not tell, or a verdict recovered from stray lines - `ignored_lines` above 0, or missing or unreadable, which is could-not-tell, as on a round recorded before L-0576 wrote the count). Otherwise own the FINDINGS with `--accept --by <who>` (only the most recent completed round, only once, never once `NEEDS_REPLAN`; a name starting `auto:` is refused), or write a new plan and get it approved — `crew_ticket.py approve` on a `NEEDS_REPLAN` ticket continues review only while a round of the ticket's three is left (a successor plan does not reopen the budget; the rounds already spent stay in the ledger and keep counting). With none left it says `0 review rounds available` and the owner's `review_ledger.py --ticket <id> --grant-round --by <you> --reason <why>` adds one; after a grant, approve a changed plan, or record the unchanged one with `review_ledger.py --successor-plan <sha256>`. With notify on, a spent budget whose last round carries a BLOCK sends the `Review out of rounds` ping when `/crew:autopilot` stops on it.

- **Symptom: a round came back `INCOMPLETE`. Check:** the `review:` lines, or `failure_class` in `.work/tickets/<id>/review.json`. An INCOMPLETE round has one of three classes. `tool` means the answer never arrived intact: a timeout, a bad or unknown exit, empty output, or a failed Codex stream. `tree` means a bundle part or web-test report changed under the reviewer. `reviewer` means the output arrived and broke the contract. Harmless prose, a heading or a code fence beside well-formed findings, with every part acknowledged and exit 0, no longer does that: the round is FINDINGS, and the ignored lines are printed on a `review: FINDINGS kept; ...` line and kept in `review.json`'s `ignored_text` (`ignored_lines` is their count). A stray line beside `CLEAN`, a misformatted contract line (`- FIX|...`, `fix|...`, a `|` table row) or a line matching the shortfall wording list ("skipped", "truncated", "could not review") is still INCOMPLETE `reviewer`. That list cannot catch every admission: one it misses ("I only inspected one of the nine files") is ignored as prose and the round stays FINDINGS, so read the ignored lines. **Fix:** a `tool` round is refunded automatically, up to three per ticket. The line reads `review: round N was a tool failure (...); refunded`. `review_run.py` retries it once by itself (L-0514), except a usage limit or a timeout; `review: retry: not retried - <why>` says when it did not. Only the failed round is given back: the rerun `/crew:review` reserves a new round, charged like any other unless it is a tool failure too, so a ticket with two charged rounds that reruns and gets FINDINGS has spent the budget. If Codex is out of quota, use the next eligible provider. A fourth tool failure on one ticket reads `NOT refunded - refund limit 3 per ticket reached` and counts. A `reviewer` or `tree` round

always counts. Rebuild the bundle (tree), or rerun and read what the reviewer wrote (reviewer).

- **Symptom: `--check-receipt` says `no accepted review receipt ... (ledger absent); could not tell whether head <sha> was reviewed`, or another host merged a PR whose review you cannot see. Cause:** the review ledger and receipt are host-local (`<git-common-dir>/crew/review/`, `.work/`), so a session on a host that did not run the review has none. That is could-not-tell, not "nothing to gate" (L-0780; PR #599 merged with 4 open BLOCKs on that misreading). **Check:**

```
python3 "<crew>/hooks/scripts/review_status.py" check --pr <PR>
```

`REVIEW_GATE ALLOW` (exit 0) means GitHub holds exactly one `crew/review` success with `blocks=0` for that exact head; `BLOCK` (1) and `COULD_NOT_TELL` (2) both refuse. The command reads only GitHub: no local ledger, no working tree. Nothing publishes the status yet (L-0821), so today it reads `COULD_NOT_TELL`, which is safe. A status attests, it does not prove, and `gh pr merge --admin` bypasses a required check.

- **Symptom: `/crew:done` refuses with a receipt error. Check:**

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --check-receipt
```

A `CLEAN` verdict writes a receipt automatically; a `FINDINGS` verdict only becomes one through `--accept`, or `--auto-accept` on a final 0-BLOCK round. An `auto-accepted` receipt stands only while its round still reads 0 BLOCK with the same lines from the same Codex or Kimi reviewer the receipt names. `--check-follow-up` fails until the follow-up's `direction.md` quotes every line verbatim (a line the receipt carries twice, twice), and on a non-UTF-8 file or an unknown receipt kind. `/crew:done` does not run it yet (L-0568 adds it to check 1), so run it yourself before closing. `--check-receipt` rebuilds the review bundle from the receipt's recorded base and fails unless the hash still matches, the receipt is for the **latest** recorded round, and the state is not `NEEDS_REPLAN` — so editing a file after the reviewer read it, or after the receipt was written, invalidates the receipt even though nothing about the ledger itself looks wrong. One exception, the delta gate: after a catch-up merge, a version bump or an anchor-only refresh, committed on a clean checkout, the receipt is kept (`receipt kept by delta gate: ...`) when the ticket's own delta is byte-identical to the reviewed one. Its stale line names what differed (`delta gate: <path> ...`, `excluded path changed`, `not clean`, `no train entry binds the integration ref` - the merge train is not armed in this clone, so the gate keeps nothing yet); a code map, rules file or diagram may move only its anchor sha after review. `excluded path changed: <path> (receipt base -> reviewed head)` in a repository that commits `.work/` notes: since L-0739 the bundle carries a committed text change there, so a receipt minted by an older crew says `outside the bundle; re-review so it is included` - run `/crew:review` again and the

new round shows it. `outside the bundle, which cannot show it` (a binary, executable, symlink or gitlink file, a force-added file the repository ignores, or a `graphify-out/` file) has one remedy: restore the path in a new commit, then re-review. Run the review from another checkout of the deploy commit, so its `review.json` and metrics row do not dirty the tree promote-gate checks. **Fix:** if the edit was deliberate, get the ticket reviewed again (spends the next round); if it was accidental, revert the edit and re-check. crew's own bookkeeping written after acceptance (the verify gate's records, a metrics row, the scope base) never stales a receipt: the bundle leaves `crew_ticket.CREW_BOOKKEEPING_PATHS` out, whatever `.gitignore` says.

- **Symptom: `/crew:done` refuses at check 2 although every Stop turn exited 0.** Stop never fails a turn on a deferral: a rule priced over `verify.stopBudgetSeconds` on its own is `chronic`, deferred to CI, and a turn where `0 rules ran` writes no verified marker (L-0710). Check 2 needs evidence for HEAD itself. **Check:**

```
python3 "<crew>/hooks/scripts/crew_status.py" --root .

python3 "<crew>/hooks/scripts/ci_receipt.py" check --root .
```

Fix: commit and push the branch so the `verify-gate` workflow runs the whole map, then rerun `/crew:done` once `CI_RECEIPT VERIFIED` names HEAD. After the PR merged, close from a branch at `origin/main`: the merge has no run of its own, and the PR head's run counts when its tree is exactly the merge's (it reads `through <sha>, which has the same tree`); if `main` moved before the merge, the trees differ and the bytes were never gated. Or run `/crew:verify --all` here, which empties the record (`verify no rules recorded`) and leaves the gate's `GATE VERIFIED` answer for this tree. An uncommitted edit after that pass turns it back to `GATE UNVERIFIED`: commit, then re-run. `UNKNOWN` (exit 3) is `gh`, the network or the artifact, never a pass; a branch that changes the gate or the workflow itself never gets a VERIFIED receipt, so it needs the local run.

- **Symptom: a receipt accepted under an older crew reads stale after the upgrade.** The upgrade to the release that brought T-0068 drops crew's bookkeeping from the bundle, so a receipt whose bundle held a non-ignored bookkeeping file (a repository that does not ignore `.crew/*`) no longer matches the rebuilt hash. **Fix:** one re-review of that ticket. Later bookkeeping writes cannot stale the new receipt.

- **Symptom: Codex hit a usage limit.** The probe printed `PROBE=limited` (exit 5) with the error on `PROBE_DETAIL=...`, or a round printed `review: codex usage limit in round N: ...`. **Cause:** Codex's usage limit, rate limit, quota or spend cap - its own message is quoted. **What happens:** that round runs on the Claude reviewer, announced as `same-family (codex limit)` and not independent; the limit is recorded in `<git-common-dir>/crew/review-limit/<ticket>.json` and applies to the next round

only. The round is spent like any other. To check Codex yourself without spending one:

```
python3 "<crew>/hooks/scripts/review_run.py" --root . --ticket <id> --scratch <dir>
\

  --provider codex --probe
```

- **Symptom: you want to send a ticket back without spending another round.**

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --reject --by
<who>
```

Refuses on a ticket already `ACCEPTED` or already `NEEDS_REPLAN`. An `ACCEPTED` ticket whose head proved unshippable takes the explicit flag, the owner's call:

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --reject --by
<who> \

  --supersede-accepted
```

It moves the ticket to `NEEDS_REPLAN`, keeps the old receipt in the `superseded` list with who and when, and clears it; only an approved successor plan continues. It refuses, changing nothing, on any other state, a name starting `auto:`, or a receipt or round it cannot read. `--by` is a recorded name, not a check of who is calling.

- **Symptom: the receipt names the wrong accepter** (a peer ran `--accept --by` under its own name for a decision the owner made).

```
python3 "<crew>/hooks/scripts/review_ledger.py" --root . --ticket <id> --correct-
acceptance \

  --by <who> --reason "<why, one line>"
```

Rewrites `accepted_by` on an `owner-accepted` receipt and appends `{round, was, now, reason, at}` to `acceptance_corrections` (shown by `--status`). Nothing else changes, so `--check-receipt` answers the same. Refuses a `clean` or `auto-accepted` receipt, a name starting `auto:`, the name already recorded, and an empty or multi-line `--by` or `--reason`. A wrong correction is fixed by another one; rows are never removed.

- **Symptom: `crew_train.py acquire` exits 1, or `/crew:review` stops with exit 10 and `review-run: train: waiting behind <ticket>`.** The clone's merge train is armed

(L-0520; the gate round takes it since L-0526, and no round was spent) and an overlapping ticket holds it, or queued first on the same base. **Check:**

```
python3 "<crew>/hooks/scripts/crew_train.py" --root . status
```

Each waiting entry lists the ticket it is behind and every colliding pair (`<mine> x <theirs>`); `touch: undeclared: <why>` means that ticket's spec has no usable `## Touch`, which overlaps everything. **Fix:** wait for the holder to land and release, then review again; fix an undeclared Touch in the spec. `merge <base> first` means the base moved in this ticket's Touch: run `crew_train.py catch-up --ticket <id>` (resolve any conflict), bump the version one past the base's, refresh the artifacts, commit, gate the merged head, and review it again if `review_ledger.py --check-receipt` reads stale: the delta gate keeps the receipt only when none of the ticket's own code moved and the clone's merge train is armed (it keeps nothing until `crew_train.py arm`). Then acquire again. `could not tell` (exit 3) means the train state could not be read — the message names the file; nothing is guessed.

- **Symptom: a lane holds the train and its session died.** `status` prints `stale?:` beside it (worktree missing, head already in the base, held for hours). Nothing releases it automatically, by design. **Fix:** once you are sure it is dead:

```
python3 "<crew>/hooks/scripts/crew_train.py" --root . release --ticket <id> --force
\

  --by <who> --reason "<why>"
```

The release is logged as a `force-release` event that the other lanes see.

- **Symptom: `check-land` says the base moved in Touch paths.** Another ticket landed changes to paths this ticket touches after it was gated, so the verdict covers a different tree. **Fix:** `crew_train.py catch-up --ticket <id>` (a merge; conflicts and rerere-replayed files are listed and left unstaged for you to inspect, `git add` and commit; a version file is never replayed and comes back conflicted), then bump the version one past the base's, refresh the artifacts, commit, gate the merged head, review it again (`/crew:review`) if `review_ledger.py --check-receipt` reads stale, then `check-land` again, so the tree the gate passed is the tree that lands. `merge-tree: HEAD conflicts with <base>` is the same fix with a conflict to resolve first.

Scope: Approval and the Completion Audit

A ticket's contract lives in `.work/tickets/<id>/`: `spec.md`'s `## Touch` section names every path the ticket may change; `plan.md`'s `Files:` lines must each fall inside Touch (steps also

carry `Lines:` and a kind tag, stated to the planner with a 300-line cap that approval does not check yet). Two hooks hold a session to that contract — see Daily workflow: scope and approval for the contract itself. This section is what goes wrong with the approval and the audit.

- **Symptom: a done ticket "has no .work/tickets/ folder". Check:** `.work/tickets/Complete/<id>/`. `crew_tracker.py archive --ticket <id>` moves a done or merged ticket's folder there (and, under Obsidian, its note to `<boardDir>/Complete/`), and `/crew:status` counts those apart (`<m> archived in Complete/`). **Fix:** none needed: every crew reader finds it there. To reopen it, move the folder and the note back by hand; `move --reopen` refuses an archived ticket.

- **Symptom: `could not tell where <id> lives`. Check:** both `.work/tickets/<id>/` and `.work/tickets/Complete/<id>/` exist (a half-finished archive, or a copy made by hand), or one of them cannot be read. **Fix:** compare the two, keep the right one, remove the stray copy. Crew never picks one.

- **Symptom: a lane worktree does not see my settings** (a CLI approval refused, `scope.mode` read as `off`, guards at their defaults). `.crew/*` is gitignored, so `git worktree add` makes a checkout with no crew config. **Check:** `/crew:status` prints `config inherited from the main checkout (<path>) ...` when the worktree reads the main checkout's `.crew/config.json` (crew 1.0.69+, T-0088), and `crew_config.py --root . --explain` starts with `repo layer: <path> (<source>)`. **Cause and fix:** a worktree with its own `.crew/config.json` or `.crew/crew.json` reads only those, never merged with the main checkout's - delete them to inherit. `/crew:status` names that case: `... the main checkout's (<path>) is not read ...`. A lane made before 1.0.69 almost always has one, a default that crew's SessionStart heal wrote there. `(unknown)` means git could not name the main checkout; then no default is written either. Of the shell and PowerShell readers (T-0096), the incident stand-down read (`_common.sh`, `promote-gate.ps1`), the cloud-guard no-python fallback and `auto-clear.sh` inherit too; in the cloud guard's fallback `unknown` counts as armed. So do the session hooks (`notify`, the handoff scripts, `context-watch`; crew 1.0.343, L-0680): a lane notifies with the main checkout's settings, and a relative handoff path still names a file in the lane. Since L-0681 the verify gate (both flavours), `review_gate.py` and the scope and completion wrappers inherit as well: an inherited `"verifyGate": false` stands a lane's Stop gate down, and with no python a lane whose main checkout has a config blocks writes and the Stop (the PowerShell 7 wrappers still allow one that strictly says `scope.mode: off`).

- **Symptom: which tickets still need my approval? Check:** `/crew:status --approvals`. It prints one ready-to-paste `/crew:approve <id>` line, with a `why: <why>` line under it, per open ticket whose approval is missing, stale or unaccepted, and nothing for merged, current or spec-only tickets (`nothing needs approval` when there are none). Approving a ticket it does not list changes nothing.

- **Symptom: an edit inside Touch is still refused. Check:** approval status.

```
python3 "<crew>/hooks/scripts/crew_ticket.py" status --ticket <id>
```

Returns `approved` (both `spec.md` and `plan.md` hash to what was approved), `stale` (either file changed since), or `none`. **Editing spec.md or plan.md after approval makes the approval stale immediately** — that is deliberate: an edited spec cannot silently widen what the guard accepts. Only a receipt from the user's own prompt counts by default; a `cli`-sourced receipt (written by `crew_ticket.py approve` directly, for tests/CI) is accepted only when `.crew/config.json` sets `scope.allowCliApproval: true`, and an `autopilot` receipt (from `crew_autopilot.py approve`) only while that is true and `autopilot.approval` still allows it. That includes a receipt written inside an `autopilot.sleep.schedule` window under `autopilot.sleep.approval`: once the window ends, the day value decides, and it may not allow it. The same holds for a manual `/crew:autopilot sleep` once its `until` passes. If `settings` prints `sleep=unknown source=manual`, `<git-common-dir>/crew/autopilot-sleep.json` could not be trusted (the warning names why): the stricter value applies per key until `/crew:autopilot wake` removes or replaces it. **Fix:** re-approve. The user types `/crew:approve <id>` again — the only other route is `/crew:autopilot` under an opted-in `autopilot.approval`; `scope_guard.py` refuses a Write/Edit under `<git-common-dir>/crew/` in every mode but `off`, so a session cannot forge or refresh its own approval.

- **Symptom: you need to touch one more path mid-ticket. Fix:** amend `spec.md`'s `## Touch` (widen it), then approve again. There is no partial-approve; amending scope is edit-then-approve, same as any other spec change.

- **Symptom: the Stop prints `outside-scope: (could not tell - ...).`** The verify gate's scope line (`scope_report.py`) reads the active ticket's approved `.work/tickets/<id>/spec.md ## Touch`, as the completion audit does. The reason names the gap: no readable `spec.md`, no `## Touch` paths, a Touch that is not approved (approve it, or approve it again after a spec edit), a broken active-ticket pointer, a pre-1.0 `.work/tickets/<id>.md` ticket (run `/crew:migrate`), or an `INDEX.md` open ticket whose `.work/tickets/<id>/` folder does not exist (fix the INDEX row or restore the folder), or an `INDEX.md` that cannot be read. A clean line can also carry `(merged main: could not tell - ...)`, for example on a detached HEAD: nothing was left out as main's, the same as the audit. Before L-0711 the line said every 1.0 ticket's file was missing. It is report-only: the refusal is `/crew:done` check 3.

- **Symptom: `/crew:done` refuses on "out of scope" for a file the edit guard never saw.** This is the Stop-time completion audit (`completion_audit.py`), not the edit guard. The edit guard only sees `Write`/`Edit`/`MultiEdit`/`NotebookEdit` tool calls; a `sed -i`, a redirect, a formatter or a `git mv` never reaches it. The audit instead diffs the **whole working tree** against the ticket's scope base (`scope_base.resolve` — the commit the ticket started from) across committed, staged, unstaged and untracked

changes, so a shell-made write is caught here even though nothing blocked it at the time. A file byte-identical to main as last merged is not counted; a `merged main: could not tell` line (a detached HEAD, none of `origin/HEAD`, `origin/main` and `main` naming a commit, or a git error) means every merged-in file was counted, so check out the ticket branch and rerun. A missing `origin/main` alone is not that: a local `main` is used instead. An untracked merged-in file (after `git rm --cached`) is main's only when `git add` would record it identically: with `core.fileMode=false` its execute bit is ignored, as git ignores it. On the review side, `diffed-from-merged=could-not-tell` (or `fork: could not tell` from `--check-receipt`) means `git merge-base <start> <merged>` gave no answer, so a file main also changed shows main's lines as the ticket's: fetch, check the start commit still exists, and rebuild. **Check it directly, without waiting for a Stop:**

```
python3 "<crew>/hooks/scripts/completion_audit.py" --check --ticket <id>
```

Fix: either the path genuinely needs to be in scope (widen Touch and re-approve), or revert the out-of-scope change. Files git ignores (including everything under `.crew/`) and `.work/` itself are outside what the audit can see at all — that is by design, not a gap to work around.

- **Symptom: the scope base is the merge-base with `main` on a repo whose branches come from `development`.** `scope_base.py --record` says "(fallback) recorded ... the merge-base with origin/main", and the review bundle or the completion audit lists hundreds of files the ticket never touched. With no `tickets.baseBranch`, the base branch is `origin/HEAD`'s target, then `origin/main`, then `main`, so a branch cut from `development` is measured against `main` and the whole integration branch looks like this ticket's change. **Fix:** set `"tickets": {"baseBranch": "development"}` in the repo's `.crew/config.json`, then `scope_base.py --root . --record <id>`. A fallback recorded against the old branch is re-derived ("re-derived ... was a merge-base guess against origin/main"); an exact record is never moved. A value that names no commit here, or a config that does not parse, reads as **could not tell**: `--record` exits 1, `--base` exits 3 with nothing on stdout, and the audit fails. It never falls back to `origin/HEAD` silently. Fix the value; do not unset it to make the error go away.
- **Symptom: the completion audit or the verify gate lists `.crew/.scope-base`, `.crew/metrics.md` or `.crew/.verify-gate.record.json`.** Only a crew from before T-0068 does that, in a repository whose `.gitignore` does not ignore `.crew/*`. **Fix:** update crew. These are crew's own bookkeeping (`crew_ticket.CREW_BOOKKEEPING_PATHS`), never a changed path for the audit, the gate or the review bundle; do not add them to Touch. A `.crew/` path the audit still lists (`.crew/verify.json`, `.crew/config.json`, a committed `.crew/incident.json` or `.crew/tfplan/` file, a session marker) is a real change: only the ticket-flow

bookkeeping and the hook logs (`.crew/guard.log`, `.crew/.autoclear.log`) are left out, never a file crew reads as a trust input.

- **`scope.mode` values, and what "auto" means:** `off` (hooks do nothing, the default), `report` (allows everything, logs the row to `.crew/guard.log`), `block` (refuses out-of-scope writes and fails the completion audit), `auto` (`report` for the first ten tickets approved in this repo, then `block`). A config file that exists but fails to parse, or names an unknown mode, resolves to `block` and says so — the unknown never collapses into the permissive value.

- **No-python fail-closed, and why it is not a bug:** if none of `python3`/`python`/`py` resolves to a *proved* working interpreter, `scope-guard.sh` (and `completion-audit.sh`, byte-for-byte the same logic) does **not** silently allow the write. It checks whether `.crew/config.json` **provably** sets `scope.mode` to `off` — a crude, non-JSON textual check, because it runs in exactly the situation where python cannot parse the file for it — and only then exits 0. Anything it cannot prove `off` (a corrupt file, `report`, `auto`, `block`, or no `scope` key at all) fails **closed** with exit 2: "no usable python ... failing closed". Fix by installing a real Python 3, not by reading the closed refusal as a false positive.

- **"the python probe timed out" (the PowerShell scope guard, completion audit, approval hook and verify gate, crew 1.1.4, L-0690).** The probe gives each candidate 3 s and the whole walk 8 s; when it runs out of time, or kills a candidate at its bound, it says "the python probe timed out" (could not tell) instead of "no usable python", with the same exit code. Whenever the probe finds nothing, `python probe:` lines follow on stderr: a summary (`found`, `not-found`, `rejected` or `timed-out`, the elapsed milliseconds and both bounds), then each candidate tried with its path, its time and what happened (`killed-at-bound`, `exit-nonzero`, `not-python-proof`, `not-tried-budget-spent`, ...). A timeout under heavy load is the machine, not a missing python: re-run when it is idle. The other PowerShell hooks and the bash twins do not report this yet.

- **`CREW_PYTHON` and the bash guards (crew 1.2.109).** The launcher exports the interpreter it already ran as `CREW_PYTHON`; `crew_py`, `crew_py_strict` and `role-write-guard.sh` take it before walking `PATH`, so a guard started by the dispatcher no longer probes every candidate again (the walk was the slow part on Windows under load). It is used only when it is an absolute path (`C:\...` is converted with `cygpath -u`), a regular executable file, named `py`, `python` or `python3.x`; anything else is ignored and the old walk runs. A guard whose helper script is missing (or whose path cannot be resolved) now refuses with `<name>.py is missing or its path could not be resolved` instead of python's `can't find '__main__' module in '<repo>'`.

Verify Gate Says 0 Rules Ran, or Deferred to CI

`verify-gate.sh` (L-0710). The Stop gate runs only the rules mapped to what changed since its marker, cheapest first, inside `verify.stopBudgetSeconds` (default 60). Two lines are expected, not errors, and neither blocks the turn:

- **`verify-gate: 0 rules ran ...`**: nothing changed, or nothing that changed has a rule Stop can run. Nothing was checked, so `.crew/.verify-verified-at` is not written or advanced. Only the diff baseline `.crew/.verify-gate.base-at` moves (when nothing committed differs from the base), so a later commit is still in scope. If that file cannot be written the turn exits 2 and says so.
- **`rules[N] is permanently over budget ... deferred to CI`** and **`NOT VERIFIED ON THIS TREE - ... deferred to CI`** (since crew 1.2.39 the quiet Stop path counts these in the verdict line's `over-budget` and `deferred` figures and writes the per-rule text to `.crew/.verify-gate.log`; `--verbose` prints it): that rule costs more than the whole budget, so Stop never runs it. The `verify-gate` workflow does, and `/crew:done` refuses until its receipt for HEAD is VERIFIED or a local run passes it. Push the branch, or run `/crew:verify --all` to check it here.

A rule that runs inside the budget and fails still exits 2, whatever was deferred beside it.

Since crew 1.3.0 the gate also keeps the **hook deadline** (`CREW_HOOK_DEADLINE`, the Stop hook's 600s limit), because the budget works from estimates and an estimate can be wrong (measured: 862s on Windows, killed by the hook, nothing recorded):

- **`verify-gate: deferred: would pass the hook deadline - rules[N] is priced Ms and Ks are left ...
- NOT VERIFIED this turn: **: before each rule, its price is held against the time left less a 60s margin. The rule did not run; the verified marker does not advance. Run/crew:verify --all` (no deadline) to check it here.
- **`a previous run was killed at the hook deadline while running <command> ... priced at Ns or more from now on`**: the last run was killed mid-rule. The gate saved what it had measured as it went (`.crew/.verify-gate.timings.json`, `.crew/.verify-gate.log` headed `RUN NOT FINISHED`), and the command in flight is now priced at no less than the time it ran, so Stop defers it by name instead of starting it into the same kill. `/crew:verify --all` measures it again.
- **`the lock at .crew/.verify-gate.lock names a holder ... whose process no longer exists`**: a killed gate's lock is reclaimed at once instead of backing off until its deadline. A holder that is alive, or that cannot be asked, still makes the next run back off (`NOTHING WAS VERIFIED`).
- **Scope larger than the branch**: the diff base is the merge-base with `origin/<default>` when that ref exists, else the local branch; the detail log (`--

`verbose`) names which. A local `main` behind the remote no longer drags merged work into scope.

Verify Gate Says UNTRUSTED, CHANGED, REMOTE CHANGED or UNKNOWN, Not Run

`verify-gate.sh` (L-0888, crew 1.3.19). `.crew/verify.json` is text the repo's authors wrote and its `run` strings reach `eval`; the Stop gate (and `--all` and `--price`) therefore runs it only for a repo this machine trusts. Otherwise it prints one line, runs nothing, writes no pass record and exits 0 (`--price` exits 1, since nothing was priced):

- **`UNTRUSTED, not run`**: nothing on this machine approves this repo. Read `.crew/verify.json` (or the smoke script the line names), then in your own terminal: `python3 <plugin>/hooks/scripts/verify_trust.py approve --root <repo>`. It shows the repo, the sha256, the remotes and the rule count, and asks for `yes`.
- **`CHANGED, not run`**: the bytes differ from every version you approved. Read the diff, then approve again.
- **`REMOTE CHANGED, not run`**: the repo is trusted by remote owner and its remotes differ from the set pinned on first sight. Check `git remote -v`, then `approve` to re-pin.
- **`UNKNOWN, not run`**: the trust store could not be read (corrupt, unreadable or locked). Fix or delete `${CLAUDE_CONFIG_DIR:-~/.claude}/crew/verify-trust.json`; an unknown is never read as trusted. It is also what you see when `git` cannot be found (the line names git), in a linked worktree, or when an allowlisted owner has to be checked. The gate does not use your PATH: it searches a fixed list only (`/mingw64/bin`, `/ucrt64/bin` and `/clangarm64/bin` on Windows; `/usr/local/bin`, `/usr/bin`, `/bin` and `/opt/homebrew/bin` elsewhere), so putting git on your own PATH changes nothing (verified: with git on the shell's PATH and absent from that list, the gate still answered UNKNOWN naming git). Install git where the gate looks (a standard Git for Windows install does), or approve the repo by hash: `approve` runs in your own terminal with your own PATH, and a hash-approved repo is recognised without git.

To trust every repo of one owner: `verify_trust.py allow-owner github.com/<owner> --under <dir>` (the `--under` limit is optional and recommended: without it a fresh directory whose `.git/config` merely names that owner is trusted on first sight). `status` shows the verdict, `revoke` removes a repo, `disallow-owner` removes an owner. A model-run `approve`, `allow-owner`, `disallow-owner` or `revoke` is refused by the scope guard: only a person at a terminal trusts a repo, and so is a verb or store path a command assembles as it runs (a variable, joined strings, a dynamic import, a call into the module's writers, `CREW_VERIFY_TRUST_FILE`, an unset `CLAUDECODE`), and a trust verb handed to a script by any name (read from stdin, a copy under another name, a wildcard path). That guard is a textual tripwire, not a

sandbox: the boundary is your OS user. The hash binds the verify map, not the scripts its rules call. Two swap windows are accepted residuals: `--price` opens its map, and `bash` reopens the smoke script, after the trust check, so a process running as you that swaps the file in that interval runs the swapped bytes (you can edit the store too). `--ci` is exempt. A project's `.claude/settings.json` `env` block reaches hook processes (measured 2026-10-25, claude 2.1.304), so a store location that is relative, or resolves inside the repo being checked (`CREW_VERIFY_TRUST_FILE`, `CLAUDE_CONFIG_DIR`, `HOME`), is refused as UNKNOWN and trusts nothing; the old `testTrustAll` store flag is gone and no store content trusts a repo without an approval or an allowlisted owner. The trust check runs as `python -E -s` under a scrubbed environment and a fixed PATH, so a `PYTHONPATH` set there cannot answer for it. Review fixes (crew 1.3.73): the trust check never takes its interpreter from an inherited `CREW_PYTHON`, and when the sanitized PATH has no python but the gate could still resolve one (a PATH entry inside the repo, or `CREW_PYTHON`) it answers UNKNOWN instead of leaving the run to the matcher; a hash approval covers the repository's top level only (a subdirectory with a byte-identical map is not trusted, and `approve` refuses to run there); `disallow-owner` clears only the pins that owner's trust made; a failed git probe inside a git directory is UNKNOWN, not a standalone path identity, so `revoke` fails loudly; `approve` refuses when git cannot read the remotes instead of storing a null pin.

Verify Gate Prints One Line, or Says "Unchanged Since Last Stop"

Since crew 1.2.39 (L-0796) the Stop gate is quiet. It prints one verdict line, then each failure exactly as before, and keeps the rest in `.crew/.verify-gate.log`:

```
verify-gate: FAILED 1 cmd | PASSED 14 cmd | NOT VERIFIED 5 item(s) (reach 2, over-budget 1, dirty tree 0, deferred 1, skipped 1, other 0) | credited 3 | budget 41s/60s | detail: .crew/.verify-gate.log
```

- **Where did the pricing, `env pinned`, `PASSED on this exact tree` and per-rule seconds lines go?** Into the log, which holds the complete text of the last run. Run the gate with `--verbose` (`-Verbose` in the PowerShell flavour) or set `CREW_VERIFY_VERBOSE=1` to get them on screen. `--ci`, `--all` and `--price` always print everything, so CI logs and `/crew:verify` look as they did.
- **Units.** FAILED, PASSED and credited count commands; NOT VERIFIED counts items: a rule for reach, over-budget, dirty tree and deferred, but a command for skipped (exit 77) and for credits withdrawn because the tree moved (under other). Do not add the two units together.
- **`NOT VERIFIED` is not `PASSED`.** A deferred, skipped, reach-excluded, over-budget or dirty-tree rule is counted under NOT VERIFIED; `other` is a rule the gate could not classify. Run `/crew:verify --all` to run them.

- **`verify-gate: unchanged since last Stop (N failing, M NOT VERIFIED): <command> | detail: <log> | ...`**: the same session is failing the same way it reported on the last Stop, so the full block is not repeated. The line names `.crew/.verify-gate.log`, which holds this run's full output. If that log cannot be written the gate never prints this line: it prints the full block instead. The turn is still blocked (exit 2). A new session, a changed failure, or deleting `.crew/.verify-gate.last-stop` prints the full block again.
- **`detail: unavailable`**: `.crew/.verify-gate.log` could not be written (a directory in its place, a read-only `.crew`). The full text is printed instead of a pointer to it.

Verify Gate Says COULD NOT TELL

`verify-gate.sh` (T-0082). A rule counts as passed only when its wrapper ended with status 0 **and** left a completion record saying the rule exited 0. Anything else that is not a plain exit status is FAILED as "could not tell": the gate prints `VERIFY FAILED: <cmd>`, then `verify-gate: COULD NOT TELL (<reason>): <cmd>`, exits 2, and neither marker advances. The summary adds `verify-gate: N rule command(s) COULD NOT BE JUDGED - counted as FAILED.` The reasons:

- **"exit status N, above 128 and not a signal number - the rule's own status"** (N from 193 to 255; signals stop at 64): the rule itself exited that code - 255 is common from `ssh` or a shell error. It is still could-not-tell, because nothing distinguishes it from a crash code.
- **"exit status N: ended by signal N-128, or the rule's own status"** (N from 129 to 192): the rule was killed (137 is KILL, 143 is TERM) - an OOM kill, an outside `timeout -s KILL`, a CI step limit - or it really exited that code. Re-run the command by hand and see which.
- **"the rule's runner ended with status N before it recorded a result"**: the wrapper around the rule was itself killed before it could write the record, so the rule's own result is unknown.
- **"no completion record"**: the wrapper ended 0 but the record is missing, empty, not 1-3 digits or above 255 - the shape a native Windows kill that reports exit 0 leaves. Nothing about the rule can be told from that run.
- **"the rule's shell could not be started"** (the PowerShell gate, deleted in L-0869, only; no current message): the resolved bash vanished or would not launch for this rule. Before T-0082 this rule silently inherited the previous rule's pass.
- **"cannot create an output-capture file or a completion-record file"** or **"no usable bash resolved"**: the existing refusals, now named as could-not-tell.
- **"the gate received TERM while this command was running"** the gate itself

was signalled mid-rule; it still exits 128+N.

The command log carries status `unknown` for such a rule; the record sync leaves that rule's entry as it was and the tree-pass cache never stores it. The gate still waits as long as a rule runs - a hung rule is not ended by the gate.

The CI receipt (`ci_receipt.py`, L-0673) lists such a command as `UNKNOWN`, and so a command the gate named failed, skipped or could-not-tell but never finished (a gate killed mid-rule). A log whose `verify-gate: <N>s total across` line does not come after its last per-rule line records `log_complete: false`; when the gate also exited non-zero, the job summary says the list is partial. The list is informative only: what the receipt accepts did not change.

A Diagram Check Fails in CI with No Reason, or "Running as Root Without --No-Sandbox"

Headless Chromium, which `mmdc` drives, refuses to start as root unless puppeteer passes `--no-sandbox`, and CI containers run as root. A hand-written `_verify` case that calls `mmdc` bare fails there, and an older `_verify/run-all.sh` or `_verify/smoke.sh` threw the check's output away, so all the log said was `FAIL <name>`.

- **Use the ready case.** Copy `plugin/crew/skills/crew-setup/templates/cases/diagrams-render.sh` into `_verify/cases/` (crew-setup does this itself in a repo that has `.mmd` files). It passes the same `--no-sandbox` puppeteer config as `plugin/crew/skills/crew-diagrams/scripts/render.sh` every time, renders to a temp directory, prints mmdc's own last lines under a failed source's `FAIL` line, and exits 77 (SKIP) where `mmdc` is not installed. `DIAGRAMS_DIR` points it at a directory other than `docs/diagrams`.
- **See why a check failed.** The current template runners print the last 5 lines of a failing check's output under its `FAIL` line and report exit 77 as `SKIP`, not a failure; a run with no failure but such a skip exits 77 itself, so crew's verify gate records it skipped, not verified. A repo set up earlier keeps its old runners: replace the counter line, the `check()` / `run()` function with the `CUR_OUT` / `cleanup` / `trap` lines above it, and the final count and exit lines from the templates. Copying `check()` alone breaks the old `_verify/smoke.sh` on the first exit 77 (`SKIP` is unset under `set -u`).

Autopilot in a Worktree, and the Refresh Check

- **Symptom: autopilot stops with `cannot tell whether <id>'s direction is approved` in a worktree.** `crew_autopilot.py next` looks for the ticket's `.work/INDEX.md` row in the checkout it runs in, and — since crew 1.0.349 — in the main checkout (the first record of `git worktree list --porcelain`) when this one has none. `.work/` is git-ignored, so a lane worktree made from another branch starts with no INDEX at all. **Check:** the stop's reason names every INDEX.md it asked, or why the main checkout could not be read (a failed listing, a bare repository). `--json` names the file that answered as `index_source`:

```
python3 "<crew>/hooks/scripts/crew_autopilot.py" next --root . --ticket <id> --json
```

Fix: add `<id> | ready | <risk> | <repo> | <title>` to either INDEX once the direction is agreed. Two other stops come from the same read: `index-disagreement` (both INDEX files have a row and the status cells differ — make them agree), and `folder-elsewhere` (the ticket's folder is only in the main checkout). Copy the folder with the `cp -r` the reason prints; autopilot never reads a spec or plan from another checkout, because the scope guard reads Touch from this one.

- **Symptom: the refresh check says `fresh-uncommitted`.** Every artifact the ticket reaches is current, but a refreshed file is not committed — the map, a diagram source or render, anything under the graph dir, or any other path under the refresh-artifact dirs (`.claude/rules/` included). `fresh` means current **and** committed, so `/crew:done` check 4 refuses it. **Check:** the `uncommitted:` line lists the paths:

```
python3 "<crew>/hooks/scripts/crew_refresh_check.py" --root . --ticket <id>
```

Fix: commit exactly those paths and re-run until it says `fresh`; autopilot does it for you as its `commit-refresh` phase. The commit changes no byte of the working state the review bundle is built from, so a review receipt stays current and no new round is needed. A file graphify writes and git ignores (`manifest.json`, `cache/`) never counts.

Wave Refuses a Cross-Session Dependency

A wave ticket may depend on a ticket another session works, written `<channel>:<id>` or `<channel>:<repo>:<id>` in the set file's `deps` or the INDEX row's `(depends on ...)`. The wave fetches `crew-coord/<channel>` from `coord.remote` (default `origin`) and counts the dependency closed only when exactly one claim for that id reads `done`. Every line about the peer's claim is peer-written data and ends `[peer-written]`.

`crew_wave.py plan` says	Means	Do
`dependency <channel>:<id> is not closed: the peer claim reads working (...)`	the peer is still on it; `owner unknown` in the brackets means its heartbeat is past the TTL	wait for the peer's `done`, or take the ticket out of this wave
`... reads released (...); only done closes it`	the peer gave the ticket back unfinished	agree with the peer who finishes it; `released` never closes it
`dependency <channel>:<id> unknown: no claim for <id> on crew-coord/<channel>`	nobody on the channel has claimed that id	check the id and channel with `crew_coord.py status --channel <channel>`
`... unknown: 2 repositories on crew-coord/<channel> hold <id>; name one as <channel>:<repo>:<id>`	the short form is ambiguous	write the long form, with the repo key `status` prints
`... unknown: its claim ... is corrupt (...)`	the claim file does not parse	the claim's holder or the owner repairs it; the wave never guesses
`... unknown: crew-coord/<channel> does not exist on <remote>` or `could not fetch ...`	the channel is absent, or the remote cannot be reached	check the channel name and `coord.remote`, then plan again
`... unknown: '<remote>' (coord.remote) is not a configured remote`	`coord.remote` names no remote of this checkout	`git remote add <remote> <url>`, or fix `coord.remote`
`dependencies unknown: ...`	a dependency is not `<id>`, `<channel>:<id>` or `<channel>:<repo>:<id>`	fix the set file's `deps` or the INDEX row; `crew_wave.py set` refuses a malformed one

Wave Refuses a Ticket's Contract

A ticket that built against a contract version (`crew_contract.py build-against`) keeps a binding in `.work/tickets/<id>/contracts.json`, naming the git remote it was built on. The wave checks each binding, on that remote, after the ticket's dependencies; run the same check alone with `crew_contract.py verify --ticket <id>` (exit 0 current, 1 changed, 3

cannot tell).

`crew_wave.py plan` says	Means	Do
`contract <n> v<N> changed since <id> built against it (...)`	the channel no longer shows that version with the bound hash, a matching body, status `built-against` and this ticket in `built_by`: someone rewrote it	do not rebind: agree a new version (`put --name <n> --new-version --ticket <new id>`) and a new ticket on each side, then build against it
`contract <n> v<N> unknown (...)`	the fetch failed, the binding's remote is no longer configured, the channel or the version's files are missing, or the record is corrupt	fix what the brackets name (remote, channel name), then plan again; nothing is assumed current
`contract bindings unknown (...)`	`contracts.json` cannot be checked, does not parse or holds no binding	restore the original file (its permissions, or a copy) if you have one; otherwise treat the contract as changed and agree a new version and a new ticket on each side. Do not delete it and run `build-against` again: that records whatever the channel holds now, a rewrite included. It is never read as "no bindings"

A newer version on the channel is information only and never refuses. The check writes nothing.

Promote Gate Blocks a Worktree Deploy

`promote-gate.sh` / `.ps1`, the `PreToolUse` hook on declared `deploy` commands. Since T-0505 it judges **the tree the deploy runs from**: the Bash call's `cwd`, moved by a leading `cd <dir> &&` and named by any `git -C <dir>`. That tree must be a worktree of the same repository, clean, and at every literal sha the command names. `.crew/verify.json`, `.work/PROMOTIONS.md` and the `.crew/.approved-<env>-<sha>` markers are still read from the session's project directory.

It is one process (crew 1.3.41). The gate used to start about forty-five (eight pythons, a dozen `git` probes), which on Git Bash took 19-20 s best case and timed out at 20 s; now

the shell starts one python and that python decides. It keeps one deadline counted from its shell's start, `promote.deadlineSeconds` (null: the hook timeout less 4 s, 16 s of 20). If a deploy still ends in "inside the gate's deadline" on a very loaded host, the deadline expired before the gate finished: that is a block, not a timeout. A number above the hook timeout less 2 s is clamped; the hook timeout itself is not a setting.

- **"the tree this deploy runs from ('...') is dirty"** names the tree it judged. If that is the main checkout while you meant a worktree, run the command there: `cd <worktree> && <deploy>`, or enter the worktree first. A dirty main checkout no longer blocks a clean worktree.
- **"has rows for sha X ..., but the newest row is not all-pass"** (L-0665): the newest `.work/PROMOTIONS.md` row for that environment and sha is a failure, a `not-run` row from `crew_ghdeploy.py record`, or anything but three `pass` cells; an older pass no longer counts. Re-run `/crew:promote <upstream>` for that sha and let it append its row.
- **"DEPLOY NOT RECORDED" for a sha no deploy ran at**: before L-0689 a command that was a FRAGMENT of a declared deploy (`git rev-parse HEAD`, `development`) matched it, and the gate wrote `.crew/.deploy-in-flight` for a deploy that never ran. A command now matches only when it contains the declared text. A stale marker left by an older crew names that sha; check that no deploy ran, then delete `.crew/.deploy-in-flight` by hand. If a real deploy is now unmatched, its runs are shorter than the declared text: declare the shortest text every real run contains.
- **"no all-pass row for sha X"** where X is the worktree's sha: the upstream environment passed a different sha. Promote the worktree's sha upstream first; a row for the main checkout's sha does not carry over.
- **"PROMOTION BLOCKED (workflow dispatch): python could not be found ..."** (L-0664, PowerShell tool): the map declares a dispatch deploy, the command names `gh` with `workflow` or `dispatches`, and no python resolved to read it. Install Python 3 or put it on PATH; this is not a pass.
- **"PROMOTION BLOCKED (workflow dispatch): the gate could not tell ..."** (T-0062 on the Bash tool, L-0664 on the PowerShell tool): the command is a `gh workflow run` or `gh api .../dispatches` and the map declares a dispatch deploy, but crew cannot read the line with certainty (a variable, a substitution, double quotes, a pipe, `--json`, `--input`, `-F k=@file`, a workflow id or display name), or a declared workflow fits no single environment. Spell it as the declared deploy is - plain literal words, the workflow's file name, the declared inputs with their declared values.
- **"the command names commit '...', but the tree it runs from ... is at ..."**: a literal `ref=<sha>` that is not the tree's HEAD. Deploy from a tree at that sha, or drop the literal.

- **"changes directory after it starts"** (a later `cd`, `bash -c 'cd ...'`, `env -C`, `make -C`), **"names more than one tree"**, **"not a form the gate reads with certainty"** or **`--git-dir`**: the gate will not guess. Put a single literal `cd <dir> &&` first, or use `git -C <dir>` - and note the directory the command itself runs in must be clean too.
- **"index entries flagged skip-worktree or assume-unchanged"**: those flags hide edits from `git status`; clear them in the tree being deployed.
- **"has uncommitted changes: it ..."** also fires when the map is deleted or untracked, or edited under skip-worktree: the gate compares the file with HEAD's copy, not with `git status`.
- **".crew/verify.json in the project dir ... has uncommitted changes"**: commit or revert the map; an uncommitted map is not policy. Since crew L-0768 this fires only when the project dir's map is the policy: the deploy runs from the project dir, or the deployed sha carries no map ("carries no committed .crew/verify.json"). A worktree deploy reads the map committed in its own sha.
- **"the map committed in the deployed sha ... does not declare it"** (L-0768): the project dir's map matched the command to an environment the deployed sha's own map lacks. Deploy a sha whose committed map declares it.
- **A waiver on the release branch is ignored** (crew before L-0768): older gates read the main checkout's map for a worktree deploy. Update crew; the waiver must be committed in the sha deployed.
- **"records the short sha X"** (L-0703): a `requires` row counts only with the full 40-character sha. Re-run the upstream promotion, or rewrite the row with `git rev-parse HEAD` of the tree that passed, after checking it is that commit.
- **"requires an accepted review of the sha being deployed"** (L-0703): no ticket's accepted receipt was of a tree identical to the one being deployed. A merge commit whose tree differs from the reviewed head (main moved, or a bump landed after the review) is not covered: deploy the reviewed head, or review the merged tree. An environment that genuinely takes unreviewed builds sets `requireReview: false` plus a `reviewReason` in the committed map. "could not tell ... did not finish" means the receipt check ran past the gate's 16s deadline; "no usable python" means the gate cannot evaluate at all - install python 3.8+.
- Never route around a block by running the deploy yourself with `!`. `/crew:promote` fixes the precondition the message names and asks you only for a `requireHuman` yes or a genuinely interactive step.

The Run Could Not Be Identified

`crew_ghdeploy.py identify` (a `github` environment under `/crew:promote`, L-0645) ends with

`result=could-not-tell reason=<code>` and exit 3. It names a run only when exactly one new run fits; it never picks among candidates.

- **`two-candidates`**: two new `workflow_dispatch` runs of that workflow, on that ref, by the same actor, appeared in the window - usually two dispatches at once. Set `correlationInput` on the entry and put that input in the workflow's `run-name`, so each run's title carries its id.
- **`none-in-timeout`**: no new run within `identifySeconds` (default 120). Check the dispatch really ran (its own Bash call, not blocked), then raise `identifySeconds` (10 to 900) if runs are slow to appear.
- **`correlation-not-found`**: new runs appeared but none had the correlation id in its title. The workflow's `run-name` must include the `correlationInput` input; there is no fallback to time.
- **`stale-prepare`**: `prepare` ran more than 600 seconds ago. Run `prepare` again, then dispatch.
- **`state-file-missing` / `state-file-unreadable`**: run `prepare` first; never edit `.crew/.ghdeploy/<env>-<N>.json` by hand.

Do not pick the run from the Actions tab and carry on: run `crew_ghdeploy.py record`, which writes `could-not-tell` and a `not-run` row, and decide from there.

Environment Guard Refusals

`env-guard.sh` / `env_guard.py`, a `PreToolUse` hook on `Bash` and `PowerShell` (L-0772). **Ships off** (`guards.profile: off`); turn it on with `{"guards": {"overrides": {"envGuard": "report"}}}` and read `.crew/guard.log` before enforcing with `"block"`. A refusal names labels only, never a value.

- **`env-dump:<form>`:** the command prints the whole environment. Read the one variable you need by name (`printenv PATH`, `echo "$HOME"`, `$env:PATH`).
- **`secret-read:<form>`:** it prints a credential-named variable. Use it without printing it (`curl -H "Authorization: Bearer $TOKEN"`), or test that it is set (`[ -n "$TOKEN" ]`).
- **`could-not-tell:<why>`:** it could not read the command well enough: an unbalanced quote, `eval "$X"`, `bash -c "$CMD"`, `eval "$(ssh-agent -s)"`, nesting deeper than 8, a globbed or brace-expanded program name (`/usr/bin/en?`, `e{n,}v`), a program from an expansion with a default (`${X:-env}`), inline code that reads a variable whose name is not a literal (`os.environ[name]`), a shell or interpreter left to read its code from stdin (`echo env | bash`, `bash < f`, a bare `python3`), a command substitution inside `(( ))`, or a non-literal path ending in `environ` (`cat $d/environ`). Write the command out literally. Under `report` these are logged, not refused.

- **Refused although the config says `off`, with no python:** the shell fallback reads the config file itself and arms the guard for a file it cannot judge: unreadable, under a directory it cannot enter, not one strict JSON object (a trailing comma, a comment, a bare word), holding a `\u` escape, or with `envGuard` set to anything but `off` anywhere in it, an explicit `null` included. Fix the file, or make python available to the hook.
- **A `/proc/*/environ` mention** outside a `git commit -m` message, a `grep` pattern or `echo` text (a `sed` script, a `find -name`) is refused too: a known false positive.

Cloud Guard False Positives

`cloud-guard.sh` / `cloud_guard.py`, a `PreToolUse` hook on `Bash` and `PowerShell`. **Ships off** (`guards.profile: off`); turn it on with `{"guards": {"overrides": {"cloudGuard": "report"}}}` first to see what it *would* refuse before enforcing with `"block"`.

- **A production-labelled environment that is not live yet keeps being refused.** Do not turn the guard off for it: list it. In `.crew/config.json` (the repo file; the machine file cannot supply it) set `{"environments": {"preProduction": {"acme-new": {"reason": "stood up in November, no customer traffic yet", "targets": ["acme-new-web1", "acme-new-db1"]}}}}`. Names and `targets` are exact, the `reason` is 10+ characters, and a malformed entry makes every environment unknown (so the refusals get stricter, not looser: the deny message says which entry is wrong). A listed name is judged non-production for a non-destroying `terraform apply` or deploy dispatch, and `prodDatabase` / `prodServer` let a command through when every production target on it is the name or one of its `targets`. A destroy, an unlisted host on the same line and every other guard are unchanged. Each use prints `crew PRE-PRODUCTION EXEMPTION` on screen and stderr and logs `env:preProduction:<name>` to `.crew/guard.log`. Remove the entry when the environment goes live (`docs/crew/config-rationale.md`, "Pre-production environments").
- **SQL text that merely looks destructive is not blocked.** `DROP`/`TRUNCATE` inside a string literal or a comment does not count: the guard strips literals and comments before matching, read both as standard SQL and as MySQL, because the two dialects disagree about what `\'` and `--` mean — a payload is flagged only if **either** reading still shows the keyword outside a literal. So `psql -c "SELECT 'DROP TABLE x' AS s"` and `psql -c 'SELECT 1 -- DROP TABLE x'` are both allowed (`plugin/crew/tests/test_cloud_guard.py`, cases `sql-literal`/`sql-comment`/`sql-block-comment`). If a query like this is still blocked, the reason (`[sqlDestructive]` in the deny message) is worth reading closely — the keyword may genuinely sit outside every literal reading.
- **A dry-run, help, or `-WhatIf` invocation is still classified as destructive.** The

guard matches on the *subcommand*, not on flags that make it a no-op: `terraform destroy --help`, `aws ec2 terminate-instances --dry-run` and `Remove-AzResourceGroup -WhatIf` are all judged exactly like the command they simulate, because nothing in `_terraform_destructive`/`_aws_destructive`/the `Remove-Az*` matcher inspects those flags. This is a known gap, not a config bug — there is no key that special-cases them. If you need to run one of these unattended, use `report` mode while testing, or approve the one-shot marker the deny message names. A literal `gh workflow run ... --help` (or `-h`) is the exception: gh prints help and dispatches nothing, so the line is not judged at all; terraform's `--help` still is.

- **A `python3`/`node` line is refused as "a terraform-family line with python code that ...".** (L-0871, crew 1.2.63; before it the denial quoted an unrelated word such as `'^JIRA_'` or `'<<'`.) With `guards.overrides.terraformApply` armed, interpreter code is refused only when it names terraform, terragrunt or tofu (in the code or anywhere on the line), or reaches a process and builds the name: the denial says which, for example `python code that spells 'terraform' out of fragments` or `builds a name at run time ('chr('). import os, import requests, os.environ.get(...)` and `node -e "console.log(process.env.X)"` alone are allowed. **Fix:** write the command plainly (`terraform apply` as its own command, where it is judged), or keep the name out of code that starts a process. Code that reads its command from the environment or a file and never names the tool is not caught (the credentials boundary is the control).

- **A workflow dispatch is refused as "could not tell" although it looks fine.** With `environments.workflows` set, a `gh workflow run` or `gh api .../dispatches` line is judged only when every word on it is a plain literal (letters, digits, `_./:=@%+,-`) or one whole single-quoted word, joined only by `;`, `&&`, `||`, `&`, newlines, `>`/`>>`/`&>`/`&>>` to a plain word and `2>&1`. A pipe, any `<`, double quotes, `$`, a glob, `bash -c '...'`, `xargs`, a command word built at run time (`$X $Y run ...`, `Start-Process $x`, an alias to a variable), gh reading stdin or a file (`--json`, `--input`, `-F k=@f`), and a malformed `environments` block in either config layer make the line could-not-tell: it asks when you are there and is denied unattended at every setting. **Fix:** write the literal form — `-f environment=staging` instead of a `--json` body, `'Deploy Staging'` instead of `"Deploy Staging"`, `> log` instead of `| tee log`, the value instead of `$ENV` — or approve the one command with the marker the refusal names (it covers those exact bytes only). Other `gh` commands (`gh pr create --title "..."`) are never gated.

- **An "unpinned" repo makes an ordinary destructive command ask or deny, even when it is obviously fine.** With no `cloud.*` pins, a read-only command (`aws s3 ls`, `az group list`) runs unchecked — but *any* destructive command has an identity the guard cannot vouch for, which is **unknown**, and unknown is never allowed unattended: it **asks** when a person is there to answer and is **denied** when not (`CREW_UNATTENDED=1`, `CI`, or a non-interactive session). This is the rule most likely

to surprise a first-time user, because it fires even in a repo that has never configured anything cloud-related. **Fix — pin the identity this repo may act as**, in `.crew/config.json` (repo-only; a machine-global `cloud` block is ignored):

```
{ "cloud": {

    "awsProfiles": ["acme-dev", "acme-sandbox-*"],

    "awsRegions": ["eu-west-1"],

    "azureSubscriptions": ["00000000-0000-0000-0000-000000000000", "Acme Dev"] } }
```

A profile/region/subscription outside the pins is denied outright (`[cloudIdentity]`). Static `AWS_ACCESS_KEY_ID` credentials and `Remove-Az*`'s saved PowerShell context are **always** unknown — the guard has no way to read either — so pinning does not silence those; only removing the static keys or naming the account through `--profile`/`--subscription` does.

Unattended Launch Refuses

`crew_unattended.py launch -- claude ...` starts an unattended session holding sealed, owner-named cloud credentials, or refuses and starts nothing (plugin README, "Unattended runs: sealed cloud credentials"). Run `python3 "<crew>/hooks/scripts/crew_unattended.py" check --root .` to see every check without launching. Each line is `ready`, `refuse` or `unknown` (could not tell, which also refuses).

- **`config: no read-only identity named`** (or `no unattendedCloud in the machine config`). The defaults name nothing. **Fix:** name the read-only role in `~/.claude/crew/config.json`, never in the repo (a repo copy is ignored and reported): set `unattendedCloud.aws.readOnly.profile` and `unattendedCloud.aws.readOnly.identity`, the assumed-role ARN prefix ending in `/`.
- **`config: environment X is not nonProd`** or **`no machine entry for nonProd environment X.`** `--environment` needs both the repo's `environments.nonProd` to match the name and a machine `unattendedCloud.aws.nonProd` entry for it. There is no production entry, by design.
- **`provider not implemented.`** Only `aws` is read; remove the other key.
- **`export: no SessionToken ...: these are static keys.`** The profile resolves to long-lived keys. **Fix:** point it at an SSO or assume-role profile; crew never runs `aws sso login` for you.
- **`export: credentials expire in under 15 minutes`** or `export ... exited`: refresh the profile's session yourself (`aws sso login`), then retry.

- **`identity: STS says ..., expected ....`** The exported credentials are a different role than the one named; a `:user/` ARN is a long-lived IAM user and always refuses. Fix the profile, or the prefix if the role was renamed.
- **`sandbox: unavailable: apply-seccomp: write /proc/self/setgroups ....`** Claude Code's sandbox starts but cannot run commands on this host (measured with `kernel.apparmor_restrict_unprivileged_userns = 1`). Every launch refuses until the host owner makes the sandbox usable or runs unattended work as a separate OS user or container.
- **`settings: <file> sets sandbox`** or **`allows Read(...).`** The repo's `.claude/settings.json` or `.claude/settings.local.json` tries to shape the sandbox or widen reads. The sealed session never loads those files, but the launcher refuses rather than trust that. **Fix:** remove the key from the repo's file, or run that work attended.
- **`settings: ... sandbox.excludedCommands`**, **`allowRead entry ... could re-open a credential store`** or **`sandbox.filesystem.disabled.`** Your `~/.claude/settings.json` loads in the sealed session and would let a command run unsandboxed or re-open a store. **Fix:** move the entry out of your user settings (or narrow the `allowRead` away from the stores), then retry.
- **`sandbox: the session can read a store: OPEN <path>.`** The sealed settings did not hold for that path (a managed or user setting may be widening it). Nothing launches; find the setting.
- **`sandbox: store not reported`**, **`could not tell`**, **`no end marker`**, **or `probe made no tool call.`** The probe did not measure every store, or its output was cut short; retry, and treat a repeat as a host problem, not a pass.
- **`refuse command: ....`** `launch` runs `claude` only (an executable file or a name on `PATH`), and supplies `--settings` and `--setting-sources` itself.

Obsidian

See Memory and Obsidian for setup. What goes wrong day to day:

- **Symptom: nothing is being captured / gardened. Check:** is the *primary* vault reachable? Capture, import and the gardener write to the primary vault only, and **never** fall back to a recall vault. If the primary is unavailable — an unmounted drive, a renamed folder — they write nothing and say why on stderr; the session itself is never blocked. `$VO adopt` shows which vault is currently `primary`.
- **Symptom: recall never surfaces anything you know is in the vault. Check:**

```
python3 "<crew plugin root>/hooks/scripts/crew_context.py" --stats
```

Read the `miss reasons` line. `cli-missing` means `obsidian-vault` is not installed where crew can find it; `cli-exit-N`, `cli-timeout`, `cli-bad-json` mean the CLI ran and failed; `no-vaults` means no vault is currently `primary` or `recall`; `no-hits` means the search itself found nothing. Only `no-hits` means "the vault genuinely has nothing relevant" — every other reason is a setup problem. Also confirm `context.enabled` is not `false` — recall and code-map injection are always on since 1.0.0 while the context hook runs, so "no recall ever appears" with the hook switched off is expected. **Fix:** `$VO recall --query "<words you expect>"` directly, to separate "the CLI can't find it" from "the hook isn't calling the CLI". An empty answer can be the CLI's relevance floor or its excluded folders, not a missing note: the obsidian-vault recall CLI drops stop words, needs more than one matching word for a longer query, matches whole words only, and never reads `wiki/sessions/archive/`. Re-run with `--min-terms 1` (the floor off) and `--include-excluded` (read the archive) and read `below_floor` and `excluded_dirs` in the `--json` output to see which one hid it.

- **Symptom: the gardener seems to be falling behind.** It runs in bounded passes on purpose, not continuously: **at most 5 items or 10 minutes per run**, whichever comes first, on **one designated host**, one run at a time. A session whose transcript lives on another machine is left queued for that machine and does not spend one of the five slots. **Check:** `$VO queue` (what is waiting) and `~/.claude/obsidian/gardener.log` for lines starting `left queued`. **Fix:** run a bounded pass by hand (`/obsidian-vault:garden` or `$VO garden-run`), or drain a large backlog in batches: `$VO drain --apply --batches 4`.

- **Symptom: two machines sharing one vault seem to be racing each other.** They are not writing the same file: each host appends to its own `inbox/pending-reflect.<host>.md` and acknowledges to its own `inbox/reflected.<host>.md`, so two machines syncing one vault never contend for the same queue file. An older `inbox/pending-reflect.md` (no host suffix) is still read if present, so an old backlog drains rather than being stranded — it is just never written again.

- **Symptom: `/crew:brainstorm` or `/crew:fix` stops because `create` refused a ticket id.** On the Obsidian ticket board, `crew_tracker.py create` refuses an id it cannot prove is yours, and prints one of four lines (paths and ids here are examples). The first three begin `id taken`, and brainstorm and fix then take the next free id on their own. For any other refusal they stop and show you the line.

    - **Another repo owns it.** `obsidian: could not update: id taken: T-0060 on Board.md belongs to another repo (https://example.invalid/team/app, per T-0060.md), not this one (https://example.invalid/other/app): give this repo its own obsidian.boardDir` **Fix:** two repos share one board. Give each repo its own `obsidian.boardDir`.

- **An older crew wrote the note's id lowercased.** `obsidian: could not update: T-0042 on Board.md belongs to another repo (https://example.invalid/team/app, per T-0042.md), not this one (https://example.invalid/Team/App): give this repo its own obsidian.boardDir; the note may carry this repo's id as an older crew wrote it (lowercased): if the card is this repo's, change the line to 'repo-id: https://example.invalid/Team/App'` **Fix:** since T-0071 an origin keeps the case of its user and path. If the card is yours, change the note's `repo-id:` line to the id it names.
- **Nobody can tell who owns it.** `obsidian: could not update: id taken: could not tell whose card T-0042 is (no T-0042.md note names its repo-id); if it is this repo's, put 'repo-id: https://example.invalid/other/app' in T-0042.md` **Fix:** if the card is yours, add the line it names to the note (see "The ticket board" in Memory and Obsidian). If it is not yours, let brainstorm take the next id.
- **Another session in this repo holds it.** `files: could not update: id taken: T-0050 is already ready "Fix tests"` **Fix:** none needed; `.work/INDEX.md` already has that ticket. This check runs before the vault is opened, so it wins even when the vault is missing.
- **Crew cannot tell which repo this is.** `obsidian: could not update: could not tell this repo's identity: git gave neither an origin URL nor a common dir for <repo path>` **Fix:** run crew inside a git checkout. `git rev-parse --git-common-dir` must answer there.

Auto Wrap-Up, Clear and Resume

See Auto wrap-up, clear and resume for the full cycle that lets a long session finish itself cleanly, clear, and pick up where it stopped — what wrap-up, auto-clear and resume each do, the machine-global keys that turn auto-clear on, and every reason a clear can refuse (logged to `.crew/.autoclear.log`).

"Auto-Clear Did Nothing on Windows"

Symptom: auto-clear is on, a handoff is written and verified, and nothing types the configured command — no keystrokes appear at all.

- **Check `context.autoClear.method`** with `/crew:config --explain` or `crew_config.py --explain`. On native Windows with no tmux pane, `"auto"` (the default) resolves to `"notify"`, not to any keystroke method — **this is not a bug**, it is the owner decision: crew does not type into a window it found itself on Windows. `"notify"` types nothing at all; instead the `Stop` hook prints a `systemMessage` telling you the handoff is written and verified and it is safe to run the configured command

yourself.

- **`sendkeys` is gone.** The Windows keystroke method was removed in crew 1.2.61 (L-0727): a config that names it is refused with "sendkeys was removed in crew 1.2.61 (L-0727); use notify or tmux", and `crew_autoclear_setup.py apply-method sendkeys` refuses by name.
- **Fix:** treat the `notify` message as the intended behaviour, or run the session inside tmux (Git Bash or WSL), where `auto` picks `tmux`. See `plugin/crew/docs/crew/config-rationale.md.

"Bare /crew:autopilot Stopped on a Goal"

Symptom: `/crew:autopilot` with no argument stops naming autopilot goals, or a `fell through:` line names a stopped goal.

With no usable handoff, a bare run looks for a running goal in `.work/autopilot/` before the active ticket (the order is argument, handoff, running goal, active ticket, INDEX; L-0659).

- **`several autopilot goals are running: a, b:`** pick one: `/crew:autopilot --goal <slug>`.
- **`could not tell whether an autopilot goal is running: .work/autopilot/<slug>.json ... could not be read:`** the file is not JSON, not an object, or its `run.state` is not `running`, `stopped` or `done`. Fix or remove it; until then the active ticket is not driven, because a goal might be running. Naming the work (`/crew:autopilot <ticket>` or `--goal <slug>`) still runs.
- **`goal <slug> stopped: <reason>`** as a `fell through:` line: a goal you or a cap stopped is never resumed by a bare run; `/crew:autopilot --goal <slug>` resumes it when you choose to.

"Claude Code Compacted by Itself"

Symptom: the session cleared or summarised itself with no `/clear` or `/compact` typed, and no message about auto-clear ran first.

That is Claude Code's own **built-in auto-compact**, not this plugin. Crew neither causes nor tunes it — `context.autoClear` and `context-watch.sh` only ever act *after* a handoff is written and verified, and nothing in this plugin can trigger a compaction on its own. There is no crew setting that changes when Claude Code's auto-compact fires.

The one thing crew does around either kind of reset: on the next `SessionStart` after any `/clear` or `/compact` — self-initiated or Claude Code's own auto-compact — `handoff-read.sh` reloads `.work/HANDOFF.md` back into context automatically, so a compaction you did not ask for still resumes from the last written handoff rather than from nothing.

Autopilot Refused a Successor Plan After an Automatic Reject

- **Symptom:** `/crew:autopilot` stops with `refused: successor plan lacks <n> of <m> finding line(s), first: <line>` after it rejected a review round itself (`autopilot.maxAutoReplans`).
- **Cause:** after an automatic reject, autopilot approves a successor plan only when `plan.md` holds every `BLOCK|` and `FIX|` line of the rejected round verbatim, each as a whole line, as many times as the round carries it (L-0670). NIT lines are not required. `could not tell: ...` means the ledger, the rejected round's row or `plan.md` could not be read.
- **Check:** `python3 plugin/crew/hooks/scripts/crew_autopilot.py replan-check --root . --ticket <id>` prints `applies= ok= missing=` and the reason, and writes nothing.
- **Fix:** quote the missing lines in the plan, or approve it yourself with `/crew:approve <id>`, which this check never blocks.

Verify Gate Says Rules Declare No `reach`

- **Symptom:** on Stop, `verify-gate: N rule(s) matched this turn declare noreach, so Stop did NOT run them` once, then on every later Stop one line: `NOT VERIFIED ON THIS TREE - N rule(s) with noreachnot run on Stop: rules[...]` (L-0733; before it, a line per rule per turn).
- **Cause:** `.crew/verify.json` predates `reach`. A rule with none that wraps a script, uses shell syntax or names a remote verb is never run unattended on Stop - it may reach a live host. The notice is shown once per map content; `.crew/.verify-gate.reach-notice` remembers which.
- **Fix:** `/crew:verify --stamp-reach` shows what each rule would get; `--apply` writes the `local` and `network` proposals, and `--set N=local|network|host` decides a wrapper or shell rule. Review the diff and commit it on its own. `/crew:verify --all` runs every rule meanwhile.
- **Symptom:** `N unverified obligation(s) belong to a rule that was edited or removed and that no rule on the same paths replaced; NOT advancing the marker [record: <path>]`. **Cause:** a rule was removed, or its `paths` changed, after its entry was recorded. A rule edited in place (same paths) supersedes its old entry instead, which is dropped with a `dropped the obligation of rules[N]` line. An undeclared-reach orphan no longer holds the marker (it is counted in the summary line); any other kind does. Entries recorded before L-0733 are never superseded. **Fix:** `/crew:verify --all` in the checkout the `[record: ...]` path names (it clears only that checkout's record, and `--ci` never prunes it), or, when that would reach `network`/`host` targets, run the `verify_record.py forget-orphans` command the line

names, from the repo root: it drops the orphans by name and runs nothing.

A Stop That Should Have Blocked Said the Hook Script Was Missing

- **Symptom:** a rule failed, but Claude Code said `Hook script appears to be missing ... Treating as non-blocking. Run /plugin to reinstall` and the turn ended anyway.
- **Cause:** Claude Code reads a Stop hook that exits 2 with empty stdout and a stderr containing `No such file` or `can't open` as a missing script. A rule like `bash _verify/check.sh` whose script is gone fails with exactly that stderr.
- **Fix:** since L-0733 crew writes `VERIFY GATE: BLOCKED (exit 2)` to stdout on every exit 2, in both flavours, so the block holds. On an older crew, update the plugin; fix or remove the rule either way.

An Agent Named in verify.json Is Not Installed

`.crew/verify.json` travels with the repo; the agents its rules name do not. A rule asking for an agent this machine lacks reviews less, and nothing in the output says so.

- **Symptom:** `/crew:status` prints `agents MISSING <name> (verify.json rule: <paths>)`, or `/crew:review` reports a requested agent as a gap. **Check:**

```
python3 "<crew>/hooks/scripts/verify_agents.py" --root . --check
```

Exit 1 lists each missing name with its rule's paths. Exit 2 (`unknown`) means a plugin registry, a settings file or `verify.json` itself would not parse, so it could not tell; it never reads that as installed. Managed-policy agents and `--agents` agents are not checked. **Fix:** install the plugin or agent, enable the plugin (`enabledPlugins`; a narrower settings scope's `false` wins), or change the rule to an agent this machine has.

Hook Processes That Spin or Hang for Hours After Their Session Ended

A hook or shell process whose parent died has no one to stop it. Measured on one Windows host: a bash guard at 27,604 CPU-seconds, a `python -` reading a program from stdin at 43,165 CPU-seconds, and 18 crew python guards blocked 4.5 to 36.7 hours at zero CPU.

- **Symptom:** a fan that never stops, or `python3` / `bash` processes years of CPU old whose parent is gone. **Check** (read-only, ends nothing):

```
python3 plugin/crew/hooks/scripts/crew_reaper.py
```

It prints one line per candidate: `would-kill` (an orphan that spins or is blocked past the limits), `report` (it has a live parent, so it is never ended; the owner chain is in the log), `skip` (protected, or it has a protected child) and `leave` (an orphan under the limits). Every decision is also a JSON line in `~/.claude/crew/reaper.log`. **Fix:** crew's own guards now end themselves at their hook timeout less 2 s (exit 124, which the dispatcher reads as the same fail-closed UNKNOWN as a timeout). For the rest, run `crew_reaper.py --apply` once, or opt in to the schedule: set `reaper.enabled` true (and `reaper.dryRun` false to let it end processes) in `~/.claude/crew/config.json`, then `crew_reaper.py --install-task --yes` on Windows or `crew_reaper.py --print-schedule` for cron and systemd `--user` text. Nothing installs the schedule for you.

The Temp Directory Fills with Crew Files

Earlier crew releases' test suites and auto-clear sender could leave files in the system temp directory: `crew-completion-audit.*` markers, `tmp.*` auto-clear sender scripts (`tmux send-keys ...` / `xdotool ...` followed by `rm -f -- <itself>`) and `tmp.*` fixture directories from crew's shell regression suite. Enough of them exhaust the inodes, and the guard and the Stop verify-gate then fail closed. Now every crew test runs with its own `TMPDIR`, the sender deletes itself before it sleeps, and the shell suite removes every fixture it makes.

- **Symptom:** `df -i /tmp` near 100%, or hooks failing on `mktemp`. **Check** (read-only):

```
cd "${TMPDIR:-/tmp}"

ls -d crew-completion-audit.* 2>/dev/null | wc -l

grep -l -e '^tmux send-keys -t' -e '^xdotool ' tmp.* 2>/dev/null | wc -l
```

Fix (an owner action; crew itself never runs it): with no crew session or test run active, remove what the check counted:

```
cd "${TMPDIR:-/tmp}"

find . -maxdepth 1 -name 'crew-completion-audit.*' -mmin +60 -delete

grep -l -e '^tmux send-keys -t' -e '^xdotool ' tmp.* 2>/dev/null | xargs -r rm -f --
```

Old shell-suite fixture directories are ordinary `tmp.*` directories (a `.crew/` and a `.git/` inside); read each before removing it.

A Setting Seems to Do Nothing

Symptom: you set a key in `.crew/config.json` or `~/.claude/crew/config.json` and crew behaves as if it were absent.

- **Check:** the `Inert settings (crew <version> does not act on them): ...` line at session start, the `inert` line in `/crew:status`, or `python3 "<crew>/hooks/scripts/crew_config.py" --root . --inert`. Each names `key=value (why)`: a ticket id means the installed crew does not implement it yet; `unknown key` means a typo or a key from another crew version; `global, repo-only` means the machine file may not set it, so it takes effect nowhere.
- **Fix:** move a repo-only key into the repo's `.crew/config.json`, correct a typo, or wait for (or install) the crew version that brings the ticket. Nothing is refused while a key is inert.

Graph Refresh Refused: Secrets-Denylisted Path

graphify reads every file its ignore rules do not exclude and records the symbols it finds in `graph.json`. For a file git tracks, `.gitignore` does not stop it: only `.graphifyignore` does. So crew checks, before it names or runs a graph build, that the root `.graphifyignore` excludes every file the secrets denylist matches.

- **Symptom:** `refresh-check` prints `graph graphify-out: unknown - graphify would read N secrets-denylisted path(s) .graphifyignore does not exclude: ...; stop - a refresh cannot settle this,` or `/crew:status` prints `graph-ignore UNCOVERED`. **Check:** which files, and from which source.

```
python3 "<crew>/hooks/scripts/crew_graph_ignore.py" --root . --check --json
```

The denylist is the built-in patterns (`.env`, `.env.*` but not `.env.example`, `*.pem`, `*.key`, `*.p12`, `*.pfx`, `id_rsa*`, `id_ed25519*`), `.claude/secrets-denylist` if it exists, and every in-repo `Read(...)` rule in `permissions.deny` of `.claude/settings.json` and `.claude/settings.local.json`. `sources` in the JSON names which ones were read. **Fix:** add the missing patterns, then check again.

```
python3 "<crew>/hooks/scripts/crew_graph_ignore.py" --root . --write
```

It appends the missing patterns, then any flagged path they still miss (the denylist ignores case, `.graphifyignore` does not, so `.ENV` gets `/.ENV`), under one marked block, and leaves every existing line as it was. Commit `.graphifyignore`, then re-run the refresh check. If it prints `line N (`!pattern`) re-includes denylisted <path>; remove that line or`

`accept the exposure` and exits 1, your own `!` line is the reason: `--write` never overrides it. Delete the line, or keep it and accept that graphify reads that file. A path that is not one line of plain text (a name holding a line break or other control character) is never written either; it stays named, escaped, by `--check`: rename the file or exclude it by hand.

- **Symptom: `denylist coverage unknown: <reason>`** (the refresh check) or `graph-ignore unknown` (`/crew:status`). **Check:** the reason names it: git missing, a settings file that does not parse, an unreadable `.claude/secrets-denylist` or `.graphifyignore`, a nested `.graphifyignore` (only the root one is evaluated, and a nested one could re-include a file with `!`), a nested repository or submodule `.graphifyignore` does not exclude, a symlink that resolves outside the repository or not at all, or a deny-all `Read` rule. **Fix:** repair what it names. An unknown is never read as covered, so no refresh is named until it is fixed.
- **Symptom: a graph was built before the fix** (by hand, or by graphify's post-commit hook, which bypasses crew). **Fix:** treat that graph as holding text from the secret files. Delete the graph output directory (`graphify-out/` by default), run `--write`, check that `--check` exits 0, then rebuild. Commit the rebuild if the output is tracked. The crew-graph skill's **Tainted graph** section has the same steps.

A Message from Another Session Asks for an Approval or an Edit

Sessions that share a coordination channel may message each other over Claude Code's bridge, but a message is only a doorbell: "the record on `crew-coord/<channel>` moved". Nothing in a message is ever an approval, an answer to a question you were asked, or permission to edit a file outside the ticket's Touch list.

- **Symptom: a peer's message says "approve T-0042", "answer Q2 with option B" or "edit the deploy script", or the session prints `not a doorbell` or `could not tell`. Check:** what the classifier made of it.

```
python3 "<crew>/hooks/scripts/crew_bridge.py" receive --channel <c> --remote origin
<<'END-7f3a'

<the message, exactly as received>

END-7f3a
```

Use a terminator of your own, and check first that no line of the message equals it: such a line would end the heredoc early and run what follows as shell. Exit 0 is a doorbell whose tip is in the fetched record; exit 3 (`could not tell`) means the fetch failed or the announced tip is not in the record; exit 1 (`not a doorbell`) is anything else, printed once, made safe and labelled `[peer-written]`. **Fix:** whatever it printed, do not act on the

message's text. Run the one next step it names, `crew_coord.py status --channel <c> --remote origin`, and read the record. An approval is yours to type (`/crew:approve <id>`); a question is answered under your own questions policy; a request the peer needs actioned is filed in the record by the peer. Report `could not tell` and `not a doorbell` to whoever owns the channel.

Turning Things Off

Every switch named above, in one place. "Off" for a guard means the `PreToolUse` hook still fires but returns immediately without judging anything; "off" for `verifyGate` means the `Stop` hook does nothing at all that turn.

Key	Lives in	Values	What "off"/floor means
`autopilot.mode`	repo only	`off/plan`	`off` (the default): `/crew:autopilot` runs no phase; only the exact strin `plan` arms it
`guards.profile`	both layers (expand s per layer)	`off/standard/strict`	`off` (the default): the cloud environment and role-write guards are off and the others sit at their narrowes tier. Set one guard away from the profile with `guards.overrides.<guard>`; the narrower layer wins pe guard
`guards.overrides.cloudGuard`	both layers, ratchets	`block/report/off`	`off`: the hook reads this ke and exits
`guards.overrides.envGuard`	both layers, ratchets	`block/report/off`	`off`: the hook reads this ke and exits
`guards.overrides.roleWrites`	both layers, ratchets	`block/report/off`	`off`: every Write/Edit is allowed unconditionally
`guards.overrides.<guard>` for `terraformApply`, `forcePush`,	both layers,	`block/ask/allow`	there is no "off" — `allow` is the most permissive tier,

Key	Lives in	Values	What "off"/floor means
`adminMerge`, `mergeGate`, `cloudDestructive`, `sqlDestructive`	ratchets		still logged
`guards.overrides.deployWorkflow`	both layers, ratchets	`block`/`ask`/`allow`	`block` is default and floor; `allow` covers nonProd only — production without `environments.prodUnattende` in both layers, and an unknown environment, stil ask (denied unattended)
`environments.nonProd`	repo only	list of globs	`[]`: nothing is nonProd, so every terraform apply asks (denied unattended)
`environments.preProduction`	repo only	object: exact name to `reason` (10+ characters) and optional exact `targets`	`{}`: nothing is exempt. A listed environment is judged non-production by the cloud guard (non-destroying apply and dispatch; `prodDatabase`/`prodServer` for its exact targets), with an audit line on screen, stderr and `.crew/guard.log`; a destroy, `block` policies and every unlisted target are unchanged
`environments.prodUnattended`	both layers, ratchets (true only when both say `true`)	bool	`false` in either layer holds i down: production applies and dispatches ask

Key	Lives in	Values	What "off"/floor means
`guards.overrides.prodDatabase`, `guards.overrides.prodServer`	both layers, ratchets	`none`/`read`/`full`	`none` is both the default and the floor
`scope.mode`	repo only	`off`/`report`/`block`/`auto`	`off`: neither the edit guard nor the completion audit runs
`scope.allowCliApproval`	repo only	bool	`false`: only a `/crew:approve` typed by the user counts (no `cli` or `autopilot` receipt
`verifyGate`	repo only	bool	`false`: the Stop verify gate does not run at all
`context.enabled`	both layers	bool	`false`: `context-watch.sh` (th Stop nag) does nothing
`context.autoClear.enabled`	both layers	bool	`false` disables the clear leg of the wrap-up/clear loop (the wrap-up itself is alway on)
`notify.provider`	both layers (a repo null inherits the global one)	`telegram`/`teams`/`none`	a repo `none` opts out even when the global file names a provider: no `question` ping, no `deploy` result (`crew_notify.py config --root .` says so)
`change.requireForProduction`	both layers, ratchets (may only turn ON)	bool	`false`: promoting needs no approved change request
`install.policy`	both layers, ratchets	see `docs/crew/config-rationale.md`,	narrowest tier refuses mor install actions

Key	Lives in	Values	What "off"/floor means
"install.policy"			

A destroy is never applied unattended, at any setting: `destroy`, `apply -destroy`, `-replace`, `workspace delete`, a saved plan whose sidecar lists a delete, or an apply with no sidecar at all asks when you are there and is denied unattended — `guards.overrides.terraformApply: allow` and `environments.prodUnattended` included. The full decision table is in `docs/crew/config-rationale.md` (`environments.*`).

`~/.claude/crew/config.json` is the machine-global layer; `.crew/config.json` is per-repo and wins where both speak. A **ratcheted** key (marked above) can only be *narrowed* by the repo relative to the machine-global value, never widened — a repo cloned from someone else cannot silently loosen a guard the machine owner set to `block`. `/crew:config --show` shows where each setting actually came from, and `/crew:config` with no argument (alias `/crew:config-setup`) sets either layer from a menu: it marks a repo value that widens a guard with `!`, and names a repo value the machine layer holds down. The same menu can delete a repo's `.crew/config.json`: it moves the file to a backup in one rename, compares it with what the preview read (a file that changed since is put back and nothing is deleted), and prints the command that restores it for sh, cmd and PowerShell.

Web Testing

Symptom: `--select web-testing` (or the ticked default row) fails, or Playwright runs but the visual rule never passes.

- **Node too old.** `install-prerequisites.{sh,ps1}` checks for Node >= 20.19 (or = 22.12) itself and refuses rather than installing or upgrading Node for you — `node -v`, install a newer one, and re-run `--select web-testing`.
- **Browsers missing system dependencies.** `npx playwright install --with-deps chromium` is what the installer runs; a bare `npx playwright install` (no `--with-deps`) skips the OS packages Chromium needs and produces a browser that launches, then crashes on first navigation.
- **Docker absent -> visual UNVERIFIED, not a fail.** `toHaveScreenshot` baselines are only meaningful when generated inside `mcr.microsoft.com/playwright:v1.63.0-noble` — the installer warns rather than blocking when Docker is missing, and the visual `verify.json` rule is written to skip (report UNVERIFIED) outside that image

rather than compare against a baseline that can never match.

- **MCP server won't connect on Windows.** Both `@playwright/mcp` and `chrome-devtools-mcp` are launched with `npx`, which the MCP host cannot invoke directly on Windows — wrap the entry as `"command": "cmd", "args": ["/c", "npx", ...]` in `.mcp.json` (documented in chrome-devtools-mcp's own troubleshooting for the "Connection closed" failure); the `stack-web` skill's `.mcp.json` example already shows this.

Windows Notes

- **"crew hooks could not tell whether Git Bash works" (crew 1.3.42).** Git Bash is there but did not answer its one-line self-test (`bash.exe -c 'exit 0'`) in time, which on a loaded Windows host (a virus scan, a build, many parallel sessions) is common: each hook has to start Git Bash, and a start can take several seconds. This is **not** a missing Git, so do not install Git; retry the same call after a few seconds. The resolver retries the self-test with backoff (1.3.41), and from 1.3.42 the dispatcher trusts the last Git Bash that passed (`~/.claude/crew/gitbash-lkg.json`: path, size and mtime, honoured only while the file is unchanged) rather than blocking a call on a slow start. It still fails closed in an armed repo when it truly cannot tell, but the message now says so. The older wording, "none was found ... Install Git for Windows", now appears only when no candidate `bin/bash.exe` exists. A third message, "found Git Bash but it failed its self-test", means the file ran and exited non-zero for a reason that is not load; repair or reinstall Git for Windows.
- **`pwsh` is not on Git Bash's `PATH`.** Every script that shells out to PowerShell names it by an absolute path or resolves it explicitly; a bare `pwsh` in a hand-typed command fails as "command not found", which is easy to misread as PowerShell itself being broken.
- **Git Bash ships without `python3`.** Every hook wrapper here resolves `python3`, then `python`, then `py`, and fails loudly on stderr rather than silently standing down when none resolves (see "no-python fail-closed", above). If a hook you expect to run says nothing at all, check for a usable Python on `PATH` first.
- **`can't open file 'D:\tmp\...\crew_autocycle.py'` (or any `.py` under `hooks/scripts`).** Git Bash rewrites a POSIX-looking script argument (`/tmp/...`, `/d/...`) into a Windows path when it starts a native `python.exe`, but only while `MSYS_NO_PATHCONV` is unset. Set, even to an empty string by a shell profile, a CI runner or a parent process, the argument reaches Python as written and Windows resolves it against the current drive. Since L-0868 every `.sh` hook except the review-harness ones (and that one record) passes its script path and its path arguments (`--root` and config data paths) through `crew_native_path` (`cygpath -m`, which needs no argument

conversion), so the variable no longer matters for them. The harness hooks (`verify-gate.sh`, `approval-hook.sh`, `scope-guard.sh`, `completion-audit.sh`, `crew-hook.sh`) the PreCompact `crew_resume.py` record in `handoff-write.sh` still depend on conversion until a follow-up lands: if one fails this way, unset `MSYS_NO_PATHCONV` for the session (`env -u MSYS_NO_PATHCONV claude`).

- **Both flavours are always registered.** `hooks.json` pairs every bash command with a `shell: "powershell"` sibling on the same event, so a machine with both `bash` and `pwsh` present runs both. This is not double-firing to fix — it is how the same hook reaches both shells; each PowerShell twin stands itself down on the wrong platform (`$env:OS -ne 'Windows_NT'`) rather than the bash twin doing that job.
- **Tests and checks are slow on Windows.** Read `/crew:status`'s `shell` line first. It names the shell route crew's jobs take (chosen automatically; `shellRoute.mode` was removed) and why. `WSL never probed` means nobody ran `crew_shell.py probe --write`; `not-installed` comes with the `wsl --install -d Ubuntu` recommendation, which crew prints and never runs. A repo on a Windows drive goes to WSL only after `crew_shell.py measure --write` has shown WSL faster there. See "Choosing the shell route on Windows" in `plugin/crew/skills/crew-setup/platform.md`.

Auto Wrap-Up, Auto-Clear and Auto-Resume

This section is folded into the troubleshooting guide (`troubleshooting.md`), linked from it as "Auto wrap-up, clear and resume". It covers the cycle that lets a long session finish itself cleanly, clear, and pick up where it stopped.

What the Cycle Does

1. **Wrap-up.** When the context window crosses its threshold, the `Stop` hook blocks once. It asks the session to finish or park the change in flight, write `.work/HANDOFF.md`, and update the ticket. This happens once for each threshold crossing in each session. It never happens on the forced continuation that follows a block. With `context.autoClear.wrapUp` armed (below), the block carries the wrap-up procedure instead, and the clear waits for its results.
2. **Clear, or notify.** After the handoff is written, auto-clear acts on it. What it does depends on `method` (below): `notify` (the default on native Windows) prints a message telling you the handoff is written and it is safe to run `/clear` (or `/compact`) yourself — it types nothing. `tmux` and `xdotool` actually type the configured command into the terminal that runs this session. Whichever method is in force, it acts only when all five of these hold: - this machine opted in; - the handoff was written by this session after the wrap-up request, is longer than a stub, and is not the automatic PreCompact skeleton; - the context reading that caused the wrap-up

was a real measurement (see "When it will not clear"); - this session is bound to its own Claude Code process, and that process has a terminal of its own (see "Which window gets the keystroke") — a headless session, such as a `claude -p` child, gets one message instead, naming its handoff and saying the process that started it must start a new one; - for a method that types (`tmux`/`xdotool`), the target window can be identified exactly, from that process — `notify` needs none of this, since it identifies no window.

Crew neither causes nor tunes Claude Code's own auto-compact. That is a separate, built-in behaviour that fires whenever Claude Code itself decides to; this cycle only reacts to a handoff once one exists, and nothing here makes auto-compact happen sooner, later, or at all. While an autopilot goal runs, the handoff names the goal, not the ticket in hand: every writer (autopilot's low-context stop, `/crew:handoff`, the PreCompact skeleton) asks `crew_autopilot.py handoff-resume`, which answers `resume: /crew:autopilot --goal <slug>` for the one running goal. Two running goals or an unreadable goal file give `resume: none`: it could not tell which goal runs, and it never falls back to the ticket form, which would drop the goal. With no usable handoff at all (a crash, a killed terminal), a bare `/crew:autopilot` looks for a running goal before the active ticket: the order is argument, handoff, running goal, active ticket, INDEX. One running goal resumes; two stop and list both; a goal file it cannot read stops (could not tell, so the active ticket is not driven); a stopped goal is named, never resumed. 3. **Resume.** The new session starts with the handoff and its next action already in context, within the 3,000-character resume budget. Nothing starts working unprompted: crew does not use `initialUserMessage`, so you press Enter to continue. Claude Code 2.1.282 drops it in an interactive session anyway (spike, 2026-09-25): it is honoured by `claude -p` only.

With **`resume.auto: true`** in the machine-global config, the handoff after `/clear` or a manual `/compact` also names the exact next command, read from the note's `resume:` line (`resume: /crew:done T-0001`): `Auto-resume: ready to run /crew:done T-0001.` — or `Auto-resume did not start: <reason>.` when a check refuses it. It does not start from the hook: you press Enter or type it, or crew types it where the terminal can be identified (below). The refusal reasons are: compact was not a manual /compact; no handoff note, or it was archived as stale, or is stale and could not be archived; the handoff is the automatic PreCompact skeleton; no resume line, `resume: none`, or a line the grammar refuses (a second line, trailing text, an unknown command, or an excluded one such as `/crew:approve`); the note's `branch:` or `head:` does not match the checkout (a ticket line only: a `--goal` line is judged by its goal file, and waits unless that goal is `running`); the ticket directory or goal file is missing; the command is not installed; `handoff-author.json` could not be read; no record of which session wrote this handoff; a later handoff write could not replace or remove `handoff-author.json` (`handoff-author.json.stuck`), or the file and its directory are both read-only so it can be neither replaced nor removed; the handoff changed since its author session wrote it; the handoff was written by another session; this session's process could not be identified (always on a host without `/proc`,

such as native Windows or macOS); the record of past auto-resumes (`resume-state.json`) could not be read, or its directory cannot be searched; this handoff was already resumed; the progress fingerprint could not be computed; the same command with no progress since the last auto-resume; `internal error` (the decision itself failed, and the handoff is still injected). Never on a plain `startup`.

Crew types it where it can (T-0013). The context hook runs the `auto-clear.sh` sender in resume mode (under Git Bash on Windows). Consent is `resume.auto`; `context.autoClear` only describes the terminal. In tmux it waits 2 seconds, polls the pane for an idle, empty input line for up to 15 seconds (both fixed), then sends the command and, half a second later, Enter. On native Windows it types nothing (`auto` there is `notify`; the `sendkeys` method was removed in crew 1.2.61). It types once per handoff (a marker under `<git-common-dir>/crew/`) and records the run first, so a run that could not be recorded is never typed. The context then says `Auto-resume: typing /crew:done T-0001 into this session in 2s (method tmux)`, or keeps the line above and adds `Auto-resume was not typed: <reason>`. Every refusal is in `.crew/.autoclear.log`: decide did not say run; no usable method, `$TMUX` unset or a pane that is not this session's; `wtype` or `xdotool`; already typed; the run could not be recorded; the input line is not empty; not ready in time; on Windows, focus lost or a tab not provable. The 2 s default comes from a measurement (input ready by 0.134 s after SessionStart on Claude Code 2.1.282); a delay is still a guess, and Windows has no probe.

A note resumes only in the session that wrote it. Write it with the Write tool (as `/crew:handoff` does): the context hook then records which session and which Claude Code process wrote it. After `/compact` the session must match; after `/clear` the process must, so a `/clear` in another terminal on the same worktree waits. A note written by Bash or by hand waits.

Turning It on and Off

Wrap-up and resume are on by default, and fixed that way: `context.autoWrapUp`, `context.autoResume` and `memory.inject` were removed, so the wrap-up instruction and the handoff injection cannot be switched off. Auto-clear is **off** by default. You turn it on per machine, in the machine-global config, because it drives this machine's keyboard:

Where	Key	Value	Effect
`~/.claude/crew/config.json` (Windows: `%USERPROFILE%\.claude\crew\`	`context.autoClear.enabled`	`true`	turns auto-clear on for every crew repo on this machine

Where	Key	Value	Effect
`config.json`)			
same	`context.autoClear.method`	`"auto"` (default), `"tmux"`, `"xdotool"`, `"notify"`, `"none"` (`"sendkeys"` was removed in crew 1.2.61 and is refused)	how the cycle acts — see "Which method, and what it does" below
same	`context.autoClear.delaySeconds`	`3`	wait before typing, so the prompt exists
same	`context.autoClear.minHandoffLines`	`5`	a shorter handoff counts as a stub
`.crew/config.json` (the repo)	`context.autoClear.enabled`	`false`	switches auto-clear off in this repo. A repo can switch it off, but `true` here does not switch it on.
`~/.claude/crew/config.json`	`resume.auto`	`true`	names the next command from the handoff's `resume:` line after `/clear` or a manual `/compact` (off by default)
`.crew/crew.json` or `.crew/config.json` (the repo)	`resume.auto`	`false`	vetoes auto-resume in this repo. `true` here does not switch it

Where	Key	Value	Effect
			on.
`~/.claude/crew/config.json`	`context.autoClear.wrapUp`	`true`	turns the wrap-up into the wrap-up procedure and makes the clear wait for its results (off by default; acts only where `enabled` is armed). A repo `false` vetoes it; a repo `true` does nothing.
`.crew/config.json`	`context.enabled`	`false`	turns off the whole context watcher, wrap-up included

`/crew:config --show` shows where each value comes from. `/crew:config` with no argument (or `/crew:config-setup`) opens a menu that sets either file from a list of values, with a dry run before anything is written. To turn the cycle off on a machine, set `context.autoClear.enabled` to `false` or delete the key. In a repo the menu offers only the veto (`false`) or `null` for `context.autoClear.enabled` and `resume.auto`, and shows `onlyRepos`/`onlySessions` read-only, because only the machine file can arm or narrow them.

Armed wrap-up (`context.autoClear.wrapUp`)

A hook cannot make the session do anything; it can only refuse to clear. So with `wrapUp` armed the threshold block sends one procedure — start no new step; run the step's `Test:` command (the active ticket's plan step, or the checks for the tracked diff with no ticket); commit only if it passes; if it cannot pass, leave the tree and write `resume: none` with the reason under **Verify first**; run `/crew:handoff --wrap-up`; end the turn — and auto-clear acts only when, beside the checks above:

- the handoff's `head:` is HEAD, so it was written after the commit;
- its `branch:` is the checked-out branch;
- no tracked file is modified (untracked files, and the handoff itself, do not count);
- its `resume:` line parses, or is `resume: none`.

Anything crew cannot tell — git failing, no `crew_resume`, on native Windows no python —

refuses. A refusal is logged, shown to you as `crew wrap-up: not clearing - <reason>`, and handed back to the session once, at its next ordinary Stop. Crew checks that the commit happened, not that the test passed: the verify gate stands down on the turn the commit is made on.

The chain end to end: wrap-up (T-0017) → the target proven from this session's own process (T-0016) → clear → `resume.auto` decides (T-0006) → the command is typed (T-0013).

Arming one scratch repo while other sessions are live

`enabled: true` arms auto-clear in **every** Claude session on the machine, not just the one you are testing. If other sessions are running, one of them can get `/clear` typed into its terminal when its own handoff lands. To limit auto-clear to one repo or one session, add these keys to the same machine file. They can only narrow what `enabled` turns on:

Key	Value	Effect
`context.autoClear.onlyRepos`	list of absolute repo paths	auto-clear runs only in these repos
`context.autoClear.onlySessions`	list of session ids	auto-clear runs only in these sessions

- A missing key or `null` does not narrow anything. An empty list `[]` turns auto-clear off everywhere. If a key holds anything other than a list, auto-clear is off everywhere too.
- When both keys are set, a session must match both.
- Paths are compared after symlinks are resolved. Backslashes and trailing separators do not matter. On Windows, case does not matter and `/c/repos/x` means `C:\repos\x`. A relative path such as `.` never matches.
- Session ids must match exactly, including case.
- Finding the repos to list: pass `--scan-root <dir>` on the one `apply-migrate` run that `/crew:migrate` makes (`crew_autoclear_setup.py --root . apply-migrate --scan-root <dir>`); a later run cannot see this repo's own opt-in, which the first run removed. It looks under `<dir>` (3 levels by default, `--scan-depth <n>`), read-only, for repos whose `.crew/config.json` or `.crew/crew.json` still says `context.autoClear.enabled: true`, and proposes them with this repo. A repo it cannot read is listed and blocks `--yes-widen`. Repos outside `<dir>`, and repos an earlier migration already converted (their opt-in is gone), cannot be found: add those by hand.
- These keys are read **only** from the machine file. If a repo's `.crew/config.json` sets them, they are ignored, so a repo cannot add itself. A repo's `enabled: false` still

turns auto-clear off.

- In any other repo or session, auto-clear stays silent. It does not write a log line, just as it does when `enabled` is off.

To arm one scratch repo safely:

```
{
  "context": {
    "autoClear": {
      "enabled": true,
      "onlyRepos": ["C:\\repos\\autoclear-scratch"]
    }
  }
}
```

On Linux, use a path such as `"/home/you/autoclear-scratch"`. When you are done testing, remove `enabled` or set it to `false`. Also delete `onlyRepos`, so that a later `enabled: true` does not arm only that repo without anyone noticing.

Which Method, and What It Does

Method	What it does	Where `auto` picks it
`tmux`	Types the command into the `$TMUX_PANE` whose process is this session's own process or an ancestor of it. No focus involved — the most reliable method.	Linux, macOS, WSL, Git Bash on Windows, when `$TMUX` names a pane and `tmux` is on PATH
`xdotool`	Types into the one X11 window owned by the nearest ancestor of this session's own process, when that window hosts no other terminal (or matching `windowTitle`).	Linux with `xdotool` on PATH and `$DISPLAY` set
`notify`	Types nothing. Prints a message saying the handoff is written and verified and it is safe to run the command	**native Windows**, whenever there is no

Method	What it does	Where `auto` picks it
	yourself. Never says anything was cleared or compacted — it does not know that, and it did not do it.	tmux pane — this is the default there, not a fallback for something missing
`sendkeys`	**Removed in crew 1.2.61 (L-0727 slice 3).** It was the Windows-only `System.Windows.Forms.SendKeys` sender in `auto-clear.ps1`, which is deleted. A config that names it is refused with "sendkeys was removed in crew 1.2.61 (L-0727); use notify or tmux".	nowhere
`none`	Refused outright.	never

When It Will Not Clear (or Notify)

Each refusal is logged with its reason to `.crew/.autoclear.log`. A Stop hook's stderr is not shown to you, so check that log first. A `notify` decline is logged the same way — "declined" rather than "refusing", since the machine has opted in and something still happened (a message, or a fallback), just not a keystroke.

Log says	Meaning
`not trustworthy (estimated-from-transcript-size)`	no usage record was found, so the reading was an estimate
`not trustworthy (unknown-window)`	the model's context window is unknown (there is no budget setting to fall back on)
`not trustworthy (stale-reading-before-compaction)`	the last reading came from before a compaction
`the wrap-up marker is empty or not JSON`	the marker was not written by this version of the hook. Nothing clears on it.
`predates this session's wrap-up request`	the handoff on disk is older than the request, probably left by an earlier session
`automatic PreCompact skeleton`	only the compaction skeleton exists, not a real handoff
`N windows have a title containing ... /`	the window is ambiguous, so nothing is

Log says	Meaning
`has N windows`	typed (`xdotool` only)
`no window belongs to any ancestor of this hook` / `... of this session's process`	the terminal's window could not be found by process id, and no `windowTitle` is set (`xdotool` only)
`sent - method notify-headless: crew: this session has no terminal of its own (...)`	a headless session (`claude -p`: entrypoint `sdk-cli`, or no controlling terminal): nothing was typed, and the message names the handoff and its `resume:` line. The process that started the session restarts it
`could not identify this session's process (...)`	no Claude Code session record names a process above the hook with this session's id and start time (or the record is unreadable, or names another session). Nothing is typed; `auto` falls back to `notify`
`could not tell whether this session has a terminal of its own (entrypoint '...' ...)`	the session record's entrypoint is not one measured to have a terminal (only `cli` is). Nothing is typed; `auto` falls back to `notify`
`passes through another Claude Code session (pid N)`	this session is a child of another session, and the pane or window above it is the parent's
`the window's owner (pid N) also hosts another terminal` / `another Claude Code session`	the terminal that owns the window also runs another tab or session, so the window is not provably this one's
`could not be listed` / `the parent of pid N could not be read` / `deeper than 16 processes`	the process tree could not be read far enough to prove anything
`found by its title alone, and another Claude Code session is live` / `has no owning process`	a `windowTitle` match (or a window with no process) cannot tell two sessions apart
`tmux pane %N could not be confirmed`	`$TMUX_PANE` is not the pane that runs this session
`method wtype ... is refused`	Wayland's `wtype` cannot target a window.

Log says	Meaning
	Use tmux instead.
`sent - method notify`	the message was printed — this is the normal, successful outcome for `notify`

Two sessions in one repository no longer interfere with each other. Wrap-up markers are `.crew/.handoff-requested-<session_id>`, and a SessionStart clears only its own session's marker.

Which Window Gets the Keystroke

Applies only to the methods that type — `tmux`, `xdotool`. `notify` identifies no window and needs none of this.

First, the session is bound to its own process: the nearest ancestor of the hook named by a Claude Code session record (`${CLAUDE_CONFIG_DIR:-~/.claude}/sessions/<pid>.json`) carrying this session's id and that process's start time. Only a session whose record says kind `interactive` and entrypoint `cli`, with a controlling terminal (native Windows has none to read), is typed into. Every rule below walks up from that process, not from the hook, and refuses when the walk passes through another Claude Code session — a child started from a session's Bash tool inherits `$TMUX`, and without this its parent's pane is the one that got `/clear`. docs/crew/config-rationale.md has the full rules and their limits (Windows' entrypoint is unmeasured; macOS cannot check the start time).

- **tmux (Linux, macOS, WSL):** the pane in `$TMUX_PANE`, and only when that pane's process is this session's own process or an ancestor of it. No title is needed. This is the most reliable method.
- **X11 (`xdotool`):** the one visible window owned by the nearest ancestor of the session's process, and only when no other terminal (another tab's tty, another session) runs under that window's owner. If that process owns more than one window, `windowTitle` has to narrow the choice to one. If no ancestor owns a window, exactly one window must match `windowTitle`, and no other session may be live, since a title cannot tell two apart. The keystroke goes to that window id after it is activated and confirmed active.